

Summary Sheet

Team Control Number: # 2625838

We study *Dancing with the Stars* (DWTS) as an inverse problem: judges' scores and weekly eliminations are observed across 34 seasons[1], but audience vote totals are withheld. The key unknown is each active contestant's weekly fan vote share, which drives eliminations under different aggregation rules.

What we built

- A reproducible season-week-contestant panel that standardizes active sets, elimination labels, and judge-score aggregates under irregular show formats (no-elimination weeks, multi-elimination weeks, varying numbers of judges, and post-exit “zero” encodings).
- A convex inference model (Task A) that reconstructs latent fan vote shares consistent with observed eliminations, and an identifiability measure (KPI2) quantifying how uniquely votes are determined each week.
- A replay engine (Tasks B/C) that re-simulates every season under percent-based versus rank-based aggregation, and faithfully implements the incumbent “combined Bottom-2 + judges' save” procedure as the baseline for controlled counterfactual comparisons.
- Mixed-effects attribution models (Task D1) separating drivers of judges' scores and fan support, and a professionalism-first institutional redesign (Task D2): a judges-only gating risk set followed by a fan-aware decision within the gate, with late-stage structured suspense mechanisms (bottom- k band / limited multi-elimination) evaluated through replay and uncertainty stress tests.

Sanity-check evidence (data integrity)

From the weekly panel we define *eligible* season-weeks (including multi-elimination weeks evaluated by bottom- k set match). Under a BL-0 baseline (judge-only with a uniform fan proxy), the elimination hit-rate is 0.364 (rank aggregation) and 0.375 (percent aggregation) across 264 eligible weeks, and the two schemes disagree on the predicted eliminated set in 3.8% of eligible weeks (FlipRate = 0.038). These rule divergences are consistent with classical observations in voting and aggregation theory.[2] These metrics validate consistent labeling and replay mechanics and serve as a baseline—not the final inferred-vote results. When counterfactual replays require post-exit inputs, we use a conservative, minimal-information imputation only to complete the replay, and we do not interpret these imputed segments as forecasting contestant performance.

Key deliverables for decision-making

- Season-by-season comparisons of how often rank vs. percent rules change eliminations and winners, and when judges' save overturns vote-driven outcomes.
- Uncertainty reports showing which weeks/contestants are non-identifiable from observed eliminations alone.
- A recommended voting policy with interpretable control knobs (judge-gated risk set, judge-fan balance within the gate, and late-stage suspense strength), selected via replay-based robustness tests under vote uncertainty.

Management recommendation

Pilot the proposed judges-gated two-stage rule in a one-season “shadow mode” (computed in parallel with the broadcast rule) and adopt a parameter region that preserves replay fidelity on eligible weeks while improving professionalism and robustness under vote-uncertainty perturbations.

Contents

1	Introduction	2
2	Problem Decomposition and Analysis	2
2.1	Problem Decomposition	3
2.2	Key Assumptions	3
2.3	Key Notations	3
2.4	Tasks and Evaluation KPIs	4
3	Data Processing and Analysis	5
3.1	Data Cleaning Principles and Validation	5
3.1.1	Cleaning Principles	5
3.1.2	Cleaning Principles	6
3.1.3	Data Cleaning Outcomes and Validation	6
3.2	Panel Construction	7
3.3	Empirical Findings from EDA (only what feeds models)	7
3.3.1	Data Visualization	7
3.3.2	Key Empirical Findings	7
3.4	Feature Engineering	9
3.5	Synthesis of Findings	9
4	Task A: Fan Vote Inference Model	9
4.1	Overview and Modeling Philosophy	9
4.2	Main Inference Model	10
4.3	Uncertainty Quantification and Sensitivity Analysis	11
4.3.1	KPI2: Vote Identifiability (interval tightness)	11
4.4	Hyperparameters, tuning, and implementation details	13
5	Task B & C: Rule Reply, Volting System, and Counterfactuals	14
5.1	Rank vs. Percent Across Seasons	15
5.1.1	Replay protocol and eligibility	15
5.1.2	Cross-season summary metrics	15
5.2	Controversial Contestants: Mechanism-driven Case Studies	16
5.3	Bottom-two and Judges' Save Mechanism	17
5.4	Synthesis of Findings from Rules	18
6	Task D-1: Impact Analysis: Pro Dancers & Celebrity Attributes	19
6.1	Outcome definitions	19
6.2	Modeling results: separating "who judges reward" vs. "who fans support"	19
6.2.1	Model 1: Judges' evaluation model	19
6.2.2	Model 2: Fans' preference model	20
6.2.3	Attribute preference gaps: judges vs. fans	20
6.2.4	Uncertainty propagation from vote inference	20
7	Task D-2: New Rule Design and Stress Testing	21
7.1	Task D-2: New Rule Design and Stress Testing	21
7.1.1	Design objective: professionalism-first, watchability as a constrained secondary goal	21
7.1.2	Bottom-2 Rule (B2 Rule): combined Bottom-2 → judges' save	21
7.1.3	Professional Bottom-2 Rule (PB2): judges-only gating → fan-aware decision within the risk set	22
7.1.4	Stress testing via replay	22
8	Conclusion	23
9	Memo to Management	23
A	Appendix: Data, Algorithms, and Reproducibility	25
B	Complete Files for Problem B	26

1 Introduction

Dancing With The Stars (DWTS) combines judges’ scores with audience voting to eliminate contestants weekly, yet the audience vote totals remain confidential.[3] This creates a recurring tension between “technical merit” and “popular support,” amplified by format changes across seasons (percent-based versus rank-based aggregation, and the later bottom-two judges’ save). The dataset [1] spans 34 seasons with contestant attributes, weekly judges’ scores, and realized eliminations, but it does not reveal audience votes—turning the central signal of fan preference into a latent variable.

Our key methodological idea is to treat DWTS as a replayable inverse problem. We first construct a canonical season–week–contestant panel with an explicit active set (excluding post-exit zero-score records and flagging special episodes). We then infer weekly fan vote *shares* by solving a constrained optimization problem: elimination-consistency constraints encode the show’s rule mechanics, slack variables absorb exceptional weeks, and a temporal smoothness prior selects a stable solution among many feasible explanations. With inferred fan support in hand, we implement a rule replay engine that simulates eliminations week-by-week under alternative aggregation schemes (rank-based vs. percent-based) and the incumbent “combined Bottom-2 + judges’ save” mechanism, enabling direct counterfactual comparisons. The inference module outputs not only point estimates $\hat{p}_{i,t}$ but also uncertainty summaries (e.g., feasible ranges and sensitivity margins). These outputs become direct inputs to the replay engine, closing the loop from latent-vote inference to counterfactual elimination paths and to quantitative evidence for rule comparison and new-system validation.

We evaluate models and voting rules using replay-based metrics that align with the show’s decisions. For vote inference, we report elimination-consistency rates, feasible-range widths (as an identifiability proxy), and sensitivity to regularization strength. For rule comparisons and new-system design, we quantify outcome divergence (who would be eliminated when), shifts in finalist composition, and robustness under stress tests that vary vote weights and the assumed rule-switch season.

To ensure interpretability, we benchmark against simple baselines (e.g., judges-only elimination and naive fan-share models) and require that any added complexity yields measurable improvements in replay fidelity and stability. Building on inferred votes, we fit mixed-effects models to separately explain judges’ scoring and fan preference, isolating professional-dancer effects and celebrity attributes. Finally, in **Task D2** we propose a professionalism-first institutional redesign: judges alone define a small “at-risk” set (gate), and fan influence is applied only within that gate; late in the season we allow structured, performance-grounded suspense (e.g., bottom- k bands or limited multi-elimination) and validate these mechanisms by replay and uncertainty stress testing.

The remainder of the paper is organized as follows: Section 2 formalizes tasks, assumptions, and notation; Section 3 describes data processing and feature construction; Sections 4–7 present vote inference, rule replay, mechanism analysis, and the proposed voting system with robustness tests; Section 8 concludes; and Section 9 provides a memo to management.

2 Problem Decomposition and Analysis

This section formalizes DWTS as a replayable inverse problem and specifies the modeling objects that close our pipeline. We define the weekly active set, the observable judges’ score signal, and the realized elimination outcomes, then introduce the latent fan vote share $p_{i,t}$ as the primary unknown. Our inference module outputs point estimates $\hat{p}_{i,t}$ together with uncertainty summaries, which feed a unified rule replay engine; this engine generates counterfactual elimination paths used for rule comparison, case-based attribution, and the validation of a proposed new voting system.

2.1 Problem Decomposition

We decompose the prompt into four linked tasks with shared inputs and a common replay-based evaluation layer.

- **Task A: Fan vote inference with uncertainty.** **Input:** weekly judges' scores and the realized eliminated set; **Output:** weekly fan vote shares $p_{i,t}$ (and uncertainty bands) over the active set; **Validation:** elimination-consistency under the assumed rule, plus identifiability diagnostics (feasible-range widths and margin-based sensitivity).
- **Task B: Voting-rule comparison via replay.** **Input:** judges' scores and inferred $p_{i,t}$; **Output:** counterfactual week-by-week eliminations under percent-based vs. rank-based aggregation (and variants); **Validation:** season-level and week-level outcome divergence measures and "fan bias" indicators.
- **Task C: Controversial contestants and judges' save attribution.** **Input:** inferred $p_{i,t}$ and replay engine; **Output:** case-study narratives supported by counterfactual trajectories with/without bottom-two judges' save; **Validation:** whether the mechanism explains observed controversies and how sensitive conclusions are to vote uncertainty.
- **Task D: Mechanism analysis and new system design.** **Input:** panel features (celebrity attributes, professional dancer identifiers) and inferred votes; **Output:** mixed-effects estimates for drivers of judges' scoring versus fan preference, and a parameterized voting rule; **Validation:** historical replay, robustness checks, and sensitivity analysis over hyperparameters and rule-switch assumptions.

2.2 Key Assumptions

We adopt a small, testable assumption budget to make the inverse problem well-posed while keeping conclusions falsifiable.

1. **Share normalization.** Fan votes are modeled as within-week shares over the active set.
2. **Rule-governed elimination.** Eliminations follow the documented aggregation rule (percent-based or rank-based), optionally augmented by bottom-two judges' save where applicable.
3. **Active-set definition and special episodes.** Post-exit "zeros" represent inactivity and are excluded from the active set; special episodes are explicitly flagged and may be accommodated via slack rather than forced exact compliance. When a counterfactual replay keeps an observed-exit contestant active, the required post-exit inputs are filled using a conservative, minimal-information imputation; this is used only to extend counterfactual trajectories and is not intended as a performance forecast.
4. **Judges as observed signal.** Judges' scores provide an observed performance signal taken as exogenous to the voting mechanism.
5. **Temporal smoothness.** Fan preference evolves smoothly week-to-week, motivating temporal regularization.

The season at which the show returns to rank-based aggregation is treated as an explicit assumption and is stress-tested using adjacent cutoffs.

2.3 Key Notations

Table 1 summarizes indices, sets, observed quantities, latent fan vote shares, and rule-dependent combined scores used throughout the inference and replay pipeline.

Symbol	Definition
s	Season index, $s = 1, \dots, 34$
t	Week index within season s
i	Contestant (celebrity–pro pair) index
$A_{s,t}$	Active contestants in season s , week t
$E_{s,t}$	Eliminated set at end of week t ; $m_{s,t} = E_{s,t} $
$j_{i,t,k}$	Score given by judge k to contestant i in week t
$J_{i,t}$	Total judges' score: $J_{i,t} = \sum_k j_{i,t,k}$ (skip missing)
$q_{i,t}^J$	Judges' share: $q_{i,t}^J = J_{i,t} / \sum_{r \in A_{s,t}} J_{r,t}$
$p_{i,t}$	Fan vote share (latent), $\sum_{i \in A_{s,t}} p_{i,t} = 1$
α	Weight on judges under percent rule (default $\alpha = 0.5$; stress-tested)
$S_{i,t}^{(P)}$	Percent combined score: $S_{i,t}^{(P)} = \alpha q_{i,t}^J + (1 - \alpha)p_{i,t}$
$R_{i,t}^J, R_{i,t}^V$	Judges rank and fan rank (descending)
$R_{i,t}^{(R)}$	Rank combined score: $R_{i,t}^{(R)} = R_{i,t}^J + R_{i,t}^V$
ξ_t	Slack for elimination-consistency; penalized in vote inference
λ	Penalty weight on slacks in vote inference objective

Table 1: Core notations used throughout the paper.

2.4 Tasks and Evaluation KPIs

Our workflow consists of four tightly coupled tasks. **Task A** reconstructs latent weekly fan vote shares $p_{i,t}$ for each active contestant by solving a season-level constrained optimization problem that enforces elimination consistency (with slack) under the documented mechanics[4, 5]. **Tasks B/C** use these inferred votes to replay each season under alternative aggregation rules (rank-based vs. percent-based, optionally with a bottom-two judges' save), producing counterfactual eliminations and placements. **Task D1** fits mixed-effects models to contrast the drivers of judges' evaluations versus audience support (celebrity attributes and professional partners). **Task D2** proposes an implementable voting rule and selects parameters via replay-based stress tests. To evaluate models and rule variants consistently, we use the following KPIs and slices.

KPI1: Elimination Set Agreement (Jaccard Similarity). On each season–week (s, t) , we compute the replayed eliminated set $\hat{E}_{s,t}$ under a specified rule and compare it to the observed eliminated-at-end-of-week set $E_{s,t}$ parsed from results. We measure set-level agreement by the Jaccard similarity

$$J_{s,t} = \frac{|E_{s,t} \cap \hat{E}_{s,t}|}{|E_{s,t} \cup \hat{E}_{s,t}|}.$$

KPI1 is reported as the week-level mean of $J_{s,t}$ over all counted season–week instances. Weeks with no elimination yield $E_{s,t} = \emptyset$ (and possibly $\hat{E}_{s,t} = \emptyset$); we adopt the convention $J_{s,t} = 1$ when both sets are empty and include such weeks in the mean.

KPI3: Rule Divergence Rate (Rank vs. Percent). Holding the same vote input (e.g., BL-0 proxy or inferred votes), KPI3 measures how often two aggregation schemes disagree on eliminations:

$$\text{KPI3} = \frac{\sum_{(s,t) \in \Omega} \mathbf{1}\{\hat{E}_{s,t}^{\text{rank}} \neq \hat{E}_{s,t}^{\text{percent}}\}}{|\Omega|}, \quad (1)$$

where Ω denotes eligible (s, t) . KPI3 is reported descriptively as “impact magnitude,” not as an optimization objective.

KPI2: Vote Identifiability (interval tightness). Because votes are unobserved, vote inference is not always identifiable.[6, 7] For each (s, t, i) , we compute a feasible interval $[p_{i,t}^{\min}, p_{i,t}^{\max}]$ over fan vote shares consistent with elimination constraints (and with a near-optimal shell around the Task A objective). We define an identifiability score as

$$ID_{i,t} = \frac{1}{p_{i,t}^{\max} - p_{i,t}^{\min} + \varepsilon}, \quad (2)$$

and summarize by quantiles across contestants, weeks, and seasons. KPI2 flags weeks and contestants where multiple vote allocations explain the observed elimination equally well.

Reproducible baseline (BL-0). Before introducing inferred votes, we report KPI1/KPI3 under BL-0 as an audit baseline. In BL-0, the rank scheme reduces to judges’ ranks only (uniform fan ranks add a constant), and the percent scheme uses judges’ percent plus a uniform fan share $1/n_{\text{active}}$. BL-0 validates denominator handling and replay mechanics, but it is not used to claim audience preference estimates.

3 Data Processing and Analysis

3.1 Data Cleaning Principles and Validation

3.1.1 Cleaning Principles

Our preprocessing is *model-driven*: we only perform transformations required to (i) construct a reproducible season–week–contestant panel, (ii) define eligibility/denominator rules for KPI computation, and (iii) enable counterfactual replay under alternative voting schemes. We preserve the dataset’s official encodings and control their impact through an explicit *active-set* definition rather than imputing within-week missing records. A conservative, minimal-information *imputation* is used *only* when a counterfactual replay keeps an observed-exit contestant active beyond the observed data horizon, solely to close the replay loop (*not* to predict future performance) [3].

Principle A — Respect official encodings (0 scores, N/A, irregular season length). The official problem statement clarifies that (i) weeks with no elimination or multiple eliminations exist, (ii) N/A can mean either “no 4th judge” or “weeks beyond a season’s actual run”, and (iii) a score of 0 is recorded for celebrities after elimination (post-exit padding). We therefore treat these as *structural encodings* rather than noise [3].

Principle B — Eligibility controls denominators (active-set construction). A contestant is considered active in (s, t) if at least one judge score exists for that week and the week does not exceed the contestant’s exit week (eliminated week parsed from `results`, or an inferred withdraw week). Post-elimination zeros are retained for auditability but excluded from denominators via `active=false`.

Principle C — Within-week aggregation only; no judge identity tracking. Because judge indices are not persistent identities across weeks/seasons, we only use within-week aggregates (weekly total, rank, and percent) and avoid modeling judge-specific time series at the preprocessing stage [3].

Principle D — Special episodes are handled explicitly. Weeks with zero eliminations are represented by the empty eliminated set and included in KPI computation via an explicit empty-set convention for Jaccard similarity; multi-elimination weeks are evaluated by set-based agreement on the bottom- k eliminated set.

3.1.2 Cleaning Principles

Our preprocessing is *model-driven*: we only perform transformations required to build a reproducible season–week–contestant panel and to support eligibility-aware KPI computation and rule replays. We adopt the following principles (write rules, not results):

1. **Respect official encodings (0 vs. N/A).** Post-elimination zeros are treated as structural padding rather than new performances; their influence is removed via the `active` definition. N/A indicates the absence of a judge in that week (e.g., only three judges) or out-of-range weeks; we aggregate with `skipna` and do not fill missing judges.
2. **Active-set driven accounting (denominator control).** All within-week percentages, replay denominators, and KPI denominators are computed on the set `active=1`. Records after a contestant exits are excluded from denominators even if they exist in the raw encoding.
3. **No aggressive within-week imputation.** During cleaning we do not impute missing judge scores and do not “invent” performance values; we only apply deterministic reshaping and aggregation. (Any “imputation” mentioned elsewhere is solely for closing counterfactual replays on unobserved post-exit segments, not for forecasting.)
4. **Special weeks are handled explicitly.** Weeks with zero eliminations and weeks with multiple eliminations are treated as first-class cases in KPI computation and replays; we do not force them into a single-elimination template.
5. **Deterministic and reproducible.** The transformation from raw wide-format inputs to the canonical replay-ready panel is deterministic; the same raw input yields the same canonical output.

3.1.3 Data Cleaning Outcomes and Validation

Outcomes (artifacts and interfaces). The output of preprocessing is a *canonical replay-ready weekly panel* (`outputs/data_cleaned/canonical_replay_ready.csv`) that contains eligibility/exit labels (`active`, `exit_type`, `exit_week`, `true_elim_flag`) and judge-based within-week features (`total_judge_score`, `judge_rank`, `judge_percent`). This panel is the single input interface used for all downstream KPI recomputation, vote-share inference, and replay simulation.

We also export two compatible variants for engineering convenience:

`outputs/data_cleaned/clean_long_data_replay_ready.csv` (replay-ready alias with aligned field names) and `outputs/data_cleaned/clean_long_data_new1.csv` (clean long panel with additional friendly fields such as `J_total`, `J_pct`, and `elim_week`).

In addition, we generate report-ready artifacts for baseline auditing:

- (i) a $\LaTeX$ table `outputs/tab/tab_baseline_consistency.tex`
- (ii) an optional figure `outputs/fig/fig_fliprate_by_season.pdf`.

Validation (what we check, and why it is sufficient for P0). We validate preprocessing through three audit layers:

- **(V1) Encoding-consistency checks.** We follow the official semantics for N/A and post-elimination zeros, preserving these encodings and excluding them from denominators via `active` [3].
- **(V2) Denominator/eligibility invariants.** `judge_percent` is computed only among active contestants within each (*season*, *week*); when total score is positive, it sums to one within the active set. The explicit `data_anomaly_zero_score` flag prevents accidental inclusion of padded zeros.
- **(V3) End-to-end reproducibility via BL-0 sanity check.** On top of the canonical panel, we implement the BL-0 baseline (rank: judge-rank only; percent: judge-percent plus uniform $1/n_{\text{active}}$) and reproduce a consistent set of KPI1/KPI3 summaries across era slices. This serves as a pipeline sanity check rather than a final model claim.

3.2 Panel Construction

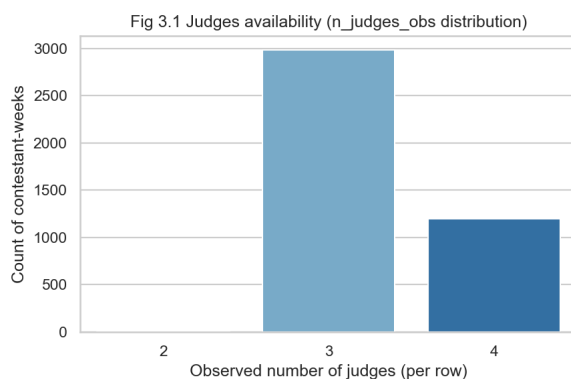
Panel construction converts the wide-format score table into a long-format dataset where each record corresponds to a unique (*season, week, celebrity*) unit. We reshape `weekX_judgeY_score` into such a long panel and compute weekly aggregates (`total_judge_score`) and within-week relative measures (`judge_rank`, `judge_percent`) among the active set. We then attach deterministic exit/eligibility labels (`exit_type`, `exit_week`, `true_elim_flag`, `active`) derived from results and score availability. The resulting canonical panel (`outputs/data_cleaned/canonical_replay_ready.csv`) is the shared input for all subsequent modules, including vote-share inference and counterfactual replay under alternative voting schemes.

3.3 Empirical Findings from EDA (only what feeds models)

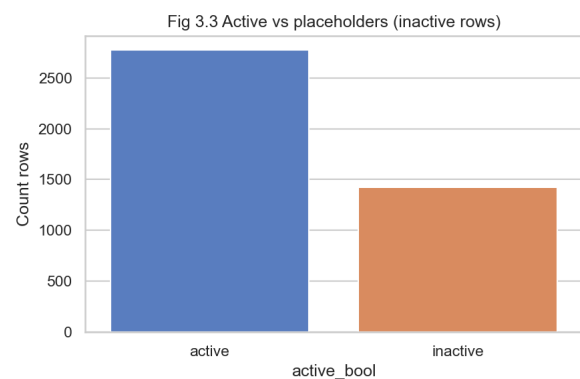
3.3.1 Data Visualization

We summarize data integrity checks and baseline consistency using:

- **(L2 figures)** preprocessing diagnostics that directly affect eligibility and KPI denominators: judges availability (Figure 1a), irregular weeks by season (Figure 2), and the presence of inactive placeholder rows (Figure 1b).
- **(Optional)** a season-level FlipRate plot (`outputs/fig/fig_fliprate_by_season.pdf`) that visualizes where rank- and percent-based replays disagree under the same inputs.



(a) Judges availability (`n_judges_obs` distribution).



(b) Active vs placeholders (`active_bool`).

Figure 1: Panel integrity diagnostics used by the preprocessing pipeline.

3.3.2 Key Empirical Findings

We report only findings that directly inform the model design and evaluation:

Finding 1 — A reproducible BL-0 baseline exists and is non-trivial. Using only judge-based information and a uniform fan proxy, BL-0 yields measurable elimination-consistency KPI1 under both rank and percent aggregation, and a non-zero flip rate between the two schemes. These quantities are used strictly as a sanity check for the pipeline and as a reference point for later vote-inference models; minor changes may occur if the canonical panel definition is updated, but the purpose is to validate consistent eligibility labeling and replay mechanics. All substantive conclusions in later sections are based on the inferred vote shares and the counterfactual replay framework, not on BL-0.

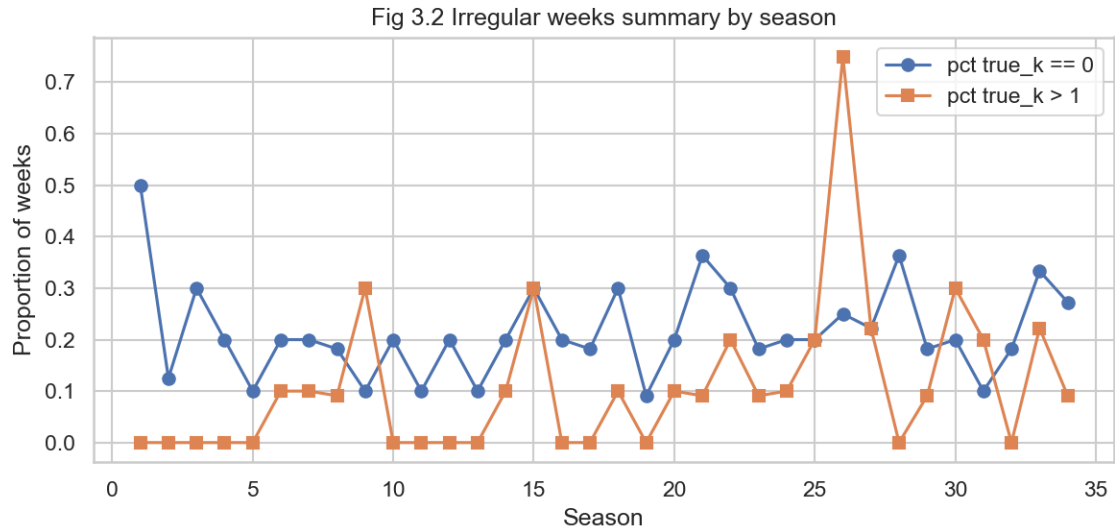


Figure 2: Irregular weeks by season: proportion of weeks with zero eliminations ($k=0$) and multiple eliminations ($k>1$).

Finding 2 — Rule comparisons must be stress-tested around the season-28 cutoff. We adopt the era split for rule-comparison slicing (Rank-era vs. Percent-era), but treat the Season-28 switch as an explicit assumption and stress-test adjacent cutoffs (27/28/29).

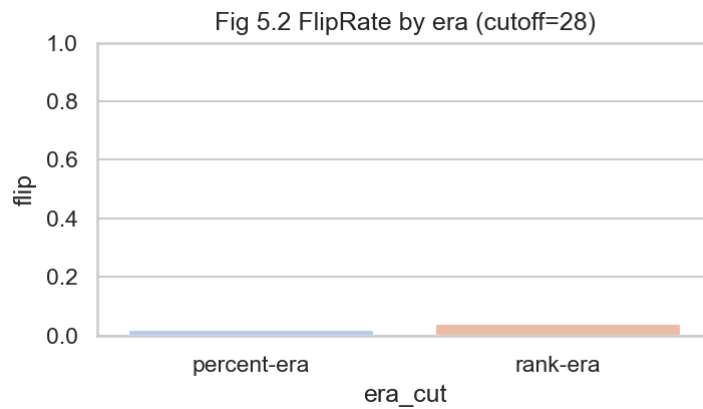


Figure 3: Cutoff at Season-28.

Finding 3 — Non-standard weeks materially affect set-based consistency metrics. Because the dataset includes weeks with no elimination or multiple eliminations, KPI1 must handle these episodes explicitly (e.g., empty-set convention for no-elimination weeks and set-based comparison for multi-elimination weeks). Such episodes can introduce non-uniform weighting under aggregation, so we interpret KPI1 primarily as a replay-pipeline consistency audit metric and emphasize relative comparisons under the same convention.

Implementation note (KPI1: Jaccard similarity, week-level mean). For each season-week (s, t) , let $E_{s,t}$ denote the observed eliminated set (possibly empty) and $\hat{E}_{s,t}$ the replayed eliminated set under a

specified rule. We measure set-level agreement by the Jaccard similarity

$$J_{s,t} = \frac{|E_{s,t} \cap \hat{E}_{s,t}|}{|E_{s,t} \cup \hat{E}_{s,t}|}.$$

KPI1 is reported as the week-level mean of $J_{s,t}$ over all counted season-week instances. Weeks with no elimination yield $E_{s,t} = \emptyset$ (and possibly $\hat{E}_{s,t} = \emptyset$), for which the ratio becomes 0/0; we adopt the convention $J_{s,t} = 1$ when both sets are empty (full agreement on “no elimination”) and include such weeks in the mean. Due to aggregation details, some no-elimination weeks may be counted more than once, introducing an additional weighting effect that can shift the absolute level of KPI1. We therefore interpret KPI1 primarily as a replay-pipeline consistency audit metric, and emphasize relative comparisons across rules and stress tests under the same convention.

3.4 Feature Engineering

We keep feature engineering minimal and aligned with downstream modeling needs:

- **Performance signals (week-level, within active set).** `total_judge_score`, `judge_rank`, and `judge_percent` are computed per (*season*, *week*) among active contestants and serve as the observed performance component when replaying elimination rules or inferring latent vote shares.
- **Explanatory covariates (contestant-level, for heterogeneity analysis).** When available, we use `ballroom_partner` (pro dancer identity), `celebrity_age_during_season`, `celebrity_industry`, and region-related fields for slicing and mixed-effects heterogeneity analyses.

3.5 Synthesis of Findings

In summary, our preprocessing produces a canonical season-week-contestant panel that (i) respects the official encodings of N/A and post-elimination zeros, (ii) enforces denominator correctness through an explicit active-set definition, and (iii) enables reproducible baseline replay consistent with our KPI definitions. Key data caveats—non-standard elimination weeks, varying judge availability, and the uncertain Season-28 era cutoff—are surfaced explicitly for robustness stress tests rather than hidden by aggressive cleaning [3].

4 Task A: Fan Vote Inference Model

We propose a deployable, data-driven fan vote inference model. The key design principle is to **treat audience votes as a latent signal and infer vote shares from observed judges’ scores and elimination outcomes under explicit rule constraints**.

4.1 Overview and Modeling Philosophy

Fan votes are a latent signal and must be inferred indirectly. In DWTS, eliminations depend on a combination of judges’ scores and audience votes, yet only judges’ scores and realized eliminations are observed. We therefore model weekly audience support as a latent vote-share vector over the active set. Because many vote configurations can be consistent with the same elimination outcome, the inference problem is generally underdetermined; our approach resolves this by (i) enforcing elimination-consistency constraints implied by the show’s rule mechanics and (ii) selecting a stable solution via temporal smoothness regularization.[6, 7] Special episodes (e.g., no-elimination or irregular-format weeks) are handled explicitly through slack variables rather than forcing exact compliance, and these slacks become a diagnostic for identifiability and model mismatch.

4.2 Main Inference Model

Decision variables. For each season s , week t , and active contestant $i \in A_{s,t}$, let $p_{i,t} \geq 0$ denote the unobserved fan vote share, with $\sum_{i \in A_{s,t}} p_{i,t} = 1$. Let $J_{i,t}$ be the observed total judges' score (sum across judges with missing values skipped) and define the judges' share $q_{i,t}^J = J_{i,t} / \sum_{r \in A_{s,t}} J_{r,t}$.

Percent-scheme combined score (primary inference target). Under the percent-based aggregation, we define the combined score

$$S_{i,t}^{(P)} = \alpha q_{i,t}^J + (1 - \alpha) p_{i,t}, \quad (3)$$

where $\alpha \in [0, 1]$ controls the relative weight of judges versus fans (default $\alpha = 0.5$; later stress-tested).

Elimination-consistency constraints with slack. Let $E_{s,t} \subseteq A_{s,t}$ denote the observed eliminated set at the end of week t with $m_{s,t} = |E_{s,t}|$. We introduce a nonnegative slack $\xi_t \geq 0$ and impose

$$S_{e,t}^{(P)} \leq S_{i,t}^{(P)} + \xi_t, \quad \forall e \in E_{s,t}, \forall i \in A_{s,t} \setminus E_{s,t}. \quad (4)$$

When the assumed rule and data are perfectly compatible, the minimum feasible ξ_t is zero; otherwise $\xi_t > 0$ quantifies the smallest violation needed to reconcile the observed outcome.[8]

Objective with judge-anchor prior. To avoid degenerate “nearly-uniform” vote reconstructions, we add a judge-anchor prior that pulls fan vote shares toward judges' score shares in the absence of strong contradictory evidence. For each season s , we solve:

$$\begin{aligned} \min_{\{p_{i,t}\}, \{\xi_t\}} \quad & \underbrace{\sum_{t=2}^{T_s} \sum_{i \in A_{s,t} \cap A_{s,t-1}} (p_{i,t} - p_{i,t-1})^2}_{\text{Temporal smoothness}} + \underbrace{\lambda_1 \sum_{t=1}^{T_s} \sum_{i \in A_{s,t}} (p_{i,t} - q_{i,t}^J)^2}_{\text{Judge anchor prior}} + \underbrace{\lambda_2 \sum_{t=1}^{T_s} \xi_t^2}_{\text{Violation penalty}} \quad (5) \\ \text{s.t.} \quad & \sum_{i \in A_{s,t}} p_{i,t} = 1, \quad p_{i,t} \geq 0, \quad \xi_t \geq 0, \quad \forall t, \\ & S_{e,t}^{(P)} \leq S_{k,t}^{(P)} + \xi_t, \quad \forall t, \forall e \in E_{s,t}, \forall k \in A_{s,t} \setminus E_{s,t}. \end{aligned}$$

Remark (objective form). Equation (5) is a convex quadratic program (QP) solved independently for each season. The smoothness term is computed only for contestants active in consecutive weeks ($A_{s,t} \cap A_{s,t-1}$), while the anchor and slack penalties apply to all active contestants and weeks. Hyperparameters (λ_1, λ_2) are selected by replay-based diagnostics (Section 4.4).[4, 5]

Algorithm 1 KPI2 computation via profile optimization (per season)

Require: Season s weekly panel: active sets $A_{s,t}$, eliminated sets $E_{s,t}$, judges' shares $q_{i,t}^J$; main-model hyperparameters $\alpha, \lambda_1, \lambda_2$; KPI2 params $\varepsilon, \delta_{\text{obj}}, \delta_\xi$; a slice selector $\mathcal{S}$ (set of (i, t) pairs to evaluate).

Ensure: Interval bounds $\{p_{i,t}^{\min}, p_{i,t}^{\max}\}$ and identifiability scores $\{\text{ID}_{i,t}\}$ for $(i, t) \in \mathcal{S}$.

- 1: **Solve main inference QP** for season s to obtain $(\hat{\mathbf{p}}, \hat{\xi})$ and $\text{OPT}_s = \mathcal{L}(\hat{\mathbf{p}}, \hat{\xi})$.
 - 2: **Define common constraints:** simplex constraints (7)–(9) and slack nonnegativity (8).
 - 3: **Add KPI2 shells:** objective shell (11) (or its SOC form) and slack cap (12).
 - 4: Choose evaluation slice $\mathcal{S}$ (e.g., late-phase weeks and/or controversial cases) to control computation.
 - 5: **for** each $(i, t) \in \mathcal{S}$ **do**
 - 6: Solve profile-min optimization (13) to get $p_{i,t}^{\min}$.
 - 7: Solve profile-max optimization (14) to get $p_{i,t}^{\max}$.
 - 8: Compute width $w_{i,t} = p_{i,t}^{\max} - p_{i,t}^{\min}$ and identifiability score $\text{ID}_{i,t} = 1/(w_{i,t} + \varepsilon)$.
 - 9: Sanity check: verify $p_{i,t}^{\min} \leq \hat{p}_{i,t} \leq p_{i,t}^{\max}$; if violated, align constraints with the main QP.
 - 10: **end for**
 - 11: Aggregate $\text{ID}_{i,t}$ by week/season/phase using medians and quantiles as defined in the KPI registry.
-

4.3 Uncertainty Quantification and Sensitivity Analysis

Uncertainty is intrinsic because the inverse problem is not fully identifiable. We report three complementary uncertainty summaries: (i) **Feasible intervals:** for each (i, t) , compute the minimum and maximum $p_{i,t}$ attainable under the same constraints (yielding an interval width as an identifiability proxy, KPI2); (ii) **Elimination margin:** quantify how much perturbation to $\hat{p}_{i,t}$ is required to flip the predicted eliminated set under the replay engine, providing a week-level stability score; and (iii) **Slack diagnostics:** ξ_t indicates weeks where observed outcomes cannot be matched under the assumed mechanics without violation; these weeks are flagged for special-episode handling and for sensitivity analysis over the rule-switch season and $\alpha, \lambda_1, \lambda_2$.

4.3.1 KPI2: Vote Identifiability (interval tightness)

KPI2: Vote Identifiability (interval tightness). The KPI registry defines vote identifiability by the tightness of a feasible interval for each (s, t, i) under an explicit elimination mechanism. In Task A, we instantiate KPI2 under the **percent-based rule** with the fan vote **share** variable $p_{i,t}$, and set $\varepsilon = 10^{-6}$ to avoid singularities. After solving the season-wise inference QP and obtaining $(\hat{p}, \hat{\xi})$ with optimal objective value OPT_s , we compute a **near-optimal feasible interval** for each active contestant-week pair by solving two auxiliary profile problems:

$$p_{i,t}^{\min} = \min p_{i,t}, \quad p_{i,t}^{\max} = \max p_{i,t},$$

subject to the original probability and elimination-consistency constraints, plus a tolerance shell that keeps solutions comparable to the fitted trajectory:

$$\mathcal{L}(p, \xi) \leq (1 + \delta_{\text{obj}})\text{OPT}_s, \quad \xi_\tau \leq \hat{\xi}_\tau + \delta_\xi \quad (\forall \tau),$$

where $\mathcal{L}(p, \xi)$ is the Task-A objective (smoothness + judge-anchor + slack penalty). The interval width $w_{i,t} = p_{i,t}^{\max} - p_{i,t}^{\min}$ is mapped to an identifiability score

$$\text{ID}_{i,t} = \frac{1}{w_{i,t} + \varepsilon}.$$

We summarize $\text{ID}_{i,t}$ by median/quantiles across contestants and weeks and aggregate by season and by early/mid/late phases; in implementation we compute KPI2 on selected slices (e.g., late-phase weeks or controversial cases) to control computational cost.

Profile optimization formulation. For a fixed season s , let T_s be the number of weeks. Define decision variables

$$\mathbf{p} = \{p_{j,\tau} : j \in A_{s,\tau}, \tau = 1, \dots, T_s\}, \quad \boldsymbol{\xi} = \{\xi_\tau : \tau = 1, \dots, T_s\}.$$

For each week τ , the percent-rule combined score is

$$S_{j,\tau}^{(P)} = \alpha q_{j,\tau}^J + (1 - \alpha)p_{j,\tau}.$$

Base constraints. We impose simplex constraints for each week:[8–10]

$$p_{j,\tau} \geq 0, \quad \forall j \in A_{s,\tau}, \forall \tau, \quad (6)$$

$$\sum_{j \in A_{s,\tau}} p_{j,\tau} = 1, \quad \forall \tau. \quad (7)$$

Slacks are nonnegative:

$$\xi_\tau \geq 0, \quad \forall \tau. \quad (8)$$

Elimination-consistency constraints (same as Task A) are

$$S_{e,\tau}^{(P)} \leq S_{k,\tau}^{(P)} + \xi_\tau, \quad \forall \tau, \forall e \in E_{s,\tau}, \forall k \in A_{s,\tau} \setminus E_{s,\tau}. \quad (9)$$

Near-optimal shell (to prevent uninformative, overly wide intervals). Let the Task-A objective be

$$\mathcal{L}(\mathbf{p}, \boldsymbol{\xi}) = \sum_{\tau=2}^{T_s} \sum_{j \in A_{s,\tau} \cap A_{s,\tau-1}} (p_{j,\tau} - p_{j,\tau-1})^2 + \lambda_1 \sum_{\tau=1}^{T_s} \sum_{j \in A_{s,\tau}} (p_{j,\tau} - q_{j,\tau}^J)^2 + \lambda_2 \sum_{\tau=1}^{T_s} \xi_\tau^2. \quad (10)$$

After solving the main inference QP, we obtain $(\hat{\mathbf{p}}, \hat{\boldsymbol{\xi}})$ and $\text{OPT}_s = \mathcal{L}(\hat{\mathbf{p}}, \hat{\boldsymbol{\xi}})$. To compute intervals around the fitted explanation, we add:

$$\mathcal{L}(\mathbf{p}, \boldsymbol{\xi}) \leq (1 + \delta_{\text{obj}}) \text{OPT}_s, \quad (11)$$

$$\xi_\tau \leq \hat{\xi}_\tau + \delta_{\xi}, \quad \forall \tau. \quad (12)$$

In implementation, (11) can be written as a convex SOC constraint by noting that $\mathcal{L}$ is a sum of squares:

$$\left\| \begin{bmatrix} \{p_{j,\tau} - p_{j,\tau-1}\}_{\tau=2..T_s, j \in A_{s,\tau} \cap A_{s,\tau-1}} \\ \{\sqrt{\lambda_1} (p_{j,\tau} - q_{j,\tau}^J)\}_{\tau=1..T_s, j \in A_{s,\tau}} \\ \{\sqrt{\lambda_2} \xi_\tau\}_{\tau=1..T_s} \end{bmatrix} \right\|_2 \leq \sqrt{(1 + \delta_{\text{obj}}) \text{OPT}_s}.$$

This SOC form is directly supported by standard convex solvers (e.g., via CVX frameworks).[11]

Profile-min and profile-max. For each active pair (i, t) with $i \in A_{s,t}$, define:

$$p_{i,t}^{\min} = \min_{\mathbf{p}, \boldsymbol{\xi}} p_{i,t} \quad (13)$$

s.t. (6) – (9), (11) – (12),

and

$$p_{i,t}^{\max} = \max_{\mathbf{p}, \boldsymbol{\xi}} p_{i,t} \quad (14)$$

s.t. (6) – (9), (11) – (12).

We then set the interval width $w_{i,t} = p_{i,t}^{\max} - p_{i,t}^{\min}$ and identifiability score $\text{ID}_{i,t} = 1/(w_{i,t} + \varepsilon)$.

Algorithm 2 KPI2 computation via profile optimization (per season)

Require: Season s weekly panel: active sets $A_{s,t}$, eliminated sets $E_{s,t}$, judges' shares $q_{i,t}^J$; main-model hyperparameters $\alpha, \lambda_1, \lambda_2$; KPI2 params $\varepsilon, \delta_{\text{obj}}, \delta_{\xi}$; a slice selector $\mathcal{S}$ (set of (i, t) pairs to evaluate).

Ensure: Interval bounds $\{p_{i,t}^{\min}, p_{i,t}^{\max}\}$ and identifiability scores $\{\text{ID}_{i,t}\}$ for $(i, t) \in \mathcal{S}$.

- 1: **Solve main inference QP** for season s to obtain $(\hat{\mathbf{p}}, \hat{\xi})$ and $\text{OPT}_s = \mathcal{L}(\hat{\mathbf{p}}, \hat{\xi})$.
- 2: **Define common constraints:** simplex constraints (7)–(9) and slack nonnegativity (8).
- 3: **Add KPI2 shells:** objective shell (11) (or its SOC form) and slack cap (12).
- 4: Choose evaluation slice $\mathcal{S}$ (e.g., late-phase weeks and/or controversial cases) to control computation.
- 5: **for** each $(i, t) \in \mathcal{S}$ **do**
- 6: Solve profile-min optimization (13) to get $p_{i,t}^{\min}$.
- 7: Solve profile-max optimization (14) to get $p_{i,t}^{\max}$.
- 8: Compute width $w_{i,t} = p_{i,t}^{\max} - p_{i,t}^{\min}$ and identifiability score $\text{ID}_{i,t} = 1/(w_{i,t} + \varepsilon)$.
- 9: Sanity check: verify $p_{i,t}^{\min} \leq \hat{p}_{i,t} \leq p_{i,t}^{\max}$; if violated, align constraints with the main QP.
- 10: **end for**
- 11: Aggregate $\text{ID}_{i,t}$ by week/season/phase using medians and quantiles as defined in the KPI registry.

4.4 Hyperparameters, tuning, and implementation details

Hyperparameters and defaults. Task A uses α (judge weight in the percent rule), λ_1 (strength of the judge-anchor prior), λ_2 (penalty on constraint violations via ξ_t), and the KPI2 parameters $\varepsilon, \delta_{\text{obj}}, \delta_{\xi}$. Given the particularity of this problem, we manually set the $\alpha \in [0.3, 0.7]$. In practice, we tested all possible values within this range at an interval of 0.01. To simplify the model and enhance its robustness, we ultimately selected $\alpha = 0.5$ for all subsequent tests and analyses. Unless otherwise stated, we set $\alpha = 0.5$ as the nominal DWTS percent weighting and treat α as a sensitivity knob in later stress tests.

We tune (λ_1, λ_2) by replay-based diagnostics. Increasing λ_2 enforces stricter elimination-consistency (smaller ξ_t) at the cost of reduced feasibility for irregular-format weeks; decreasing λ_2 allows the model to absorb format mismatches but may underfit eliminations. Increasing λ_1 pulls inferred votes toward judges' score shares and helps avoid near-uniform solutions, while excessively large λ_1 may suppress genuine judge–fan disagreements that are evidenced by the observed eliminations. Practically, we tune (λ_1, λ_2) on a season-by-season basis by balancing (i) the frequency and magnitude of nonzero slacks $\hat{\xi}_t$, (ii) the stability of inferred top- k vote shares across adjacent weeks, and (iii) downstream replay fidelity in Task B.

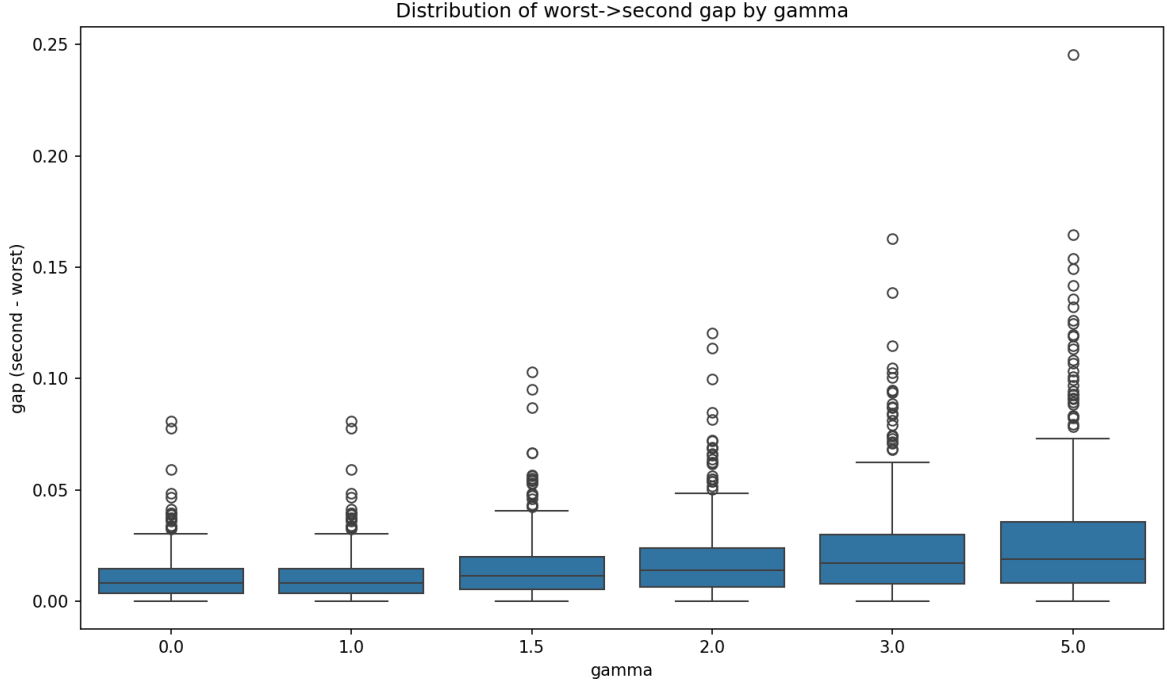


Figure 4: KPI2 identifiability scores by phase across all seasons under the percent rule with $\alpha = 0.5$.

Special episodes and multi-elimination weeks. Our formulation supports (a) multi-elimination weeks by allowing $|E_{s,t}| > 1$ and enforcing the same pairwise inequalities against all non-eliminated contestants, and (b) non-elimination or irregular weeks by allowing $E_{s,t} = \emptyset$ (no elimination constraints) while still solving the smoothness objective; such weeks can also be explicitly flagged and assessed via slack and sensitivity tests. If a season contains weeks whose outcomes cannot be reconciled with the assumed mechanics, the model will surface this via persistently large $\hat{\xi}_t$; these weeks are treated as format exceptions rather than forcing ad hoc re-labeling.

Solver and numerical stability. The main inference problem is a convex QP and can be solved season-wise using standard QP solvers. The KPI2 shell can be implemented either directly as a quadratic constraint or in SOC form; the latter is solver-friendly in CVX frameworks. To avoid numerical issues, we recommend (i) normalizing judges' scores into shares q^J (already done), (ii) using $\varepsilon = 10^{-6}$ for identifiability scoring, and (iii) keeping δ_{obj} small (e.g., 1–5%) so profile intervals remain comparable to the fitted explanation.

5 Task B & C: Rule Reply, Volting System, and Counterfactuals

Following the official rule definitions, we perform season-week counterfactual replays under (i) rank-based aggregation and (ii) percent-based aggregation. We initialize each replay with the observed eligibility labels, but update the active set week-by-week according to the counterfactual eliminations under the chosen rule. When a counterfactual path keeps an observed-exit contestant active beyond the observed data horizon, we apply a conservative, minimal-information imputation solely to complete the replay (not to forecast future performance). As a reproducible sanity-check baseline (BL-0), we use judge-only information with a uniform fan proxy and report elimination hit-rate (KPI1) and the rule divergence rate (KPI3/FlipRate). This chapter summarizes cross-season differences (Section 5.1), explains divergences via controversial case studies (Section 5.2), evaluates the incremental impact of bottom-two and judges' save (Section 5.3), and synthesizes implications (Section 5.4).

5.1 Rank vs. Percent Across Seasons

Results in this section are first reported under BL-0 as a reproducible sanity check; Task A inferred votes are used in subsequent replays.

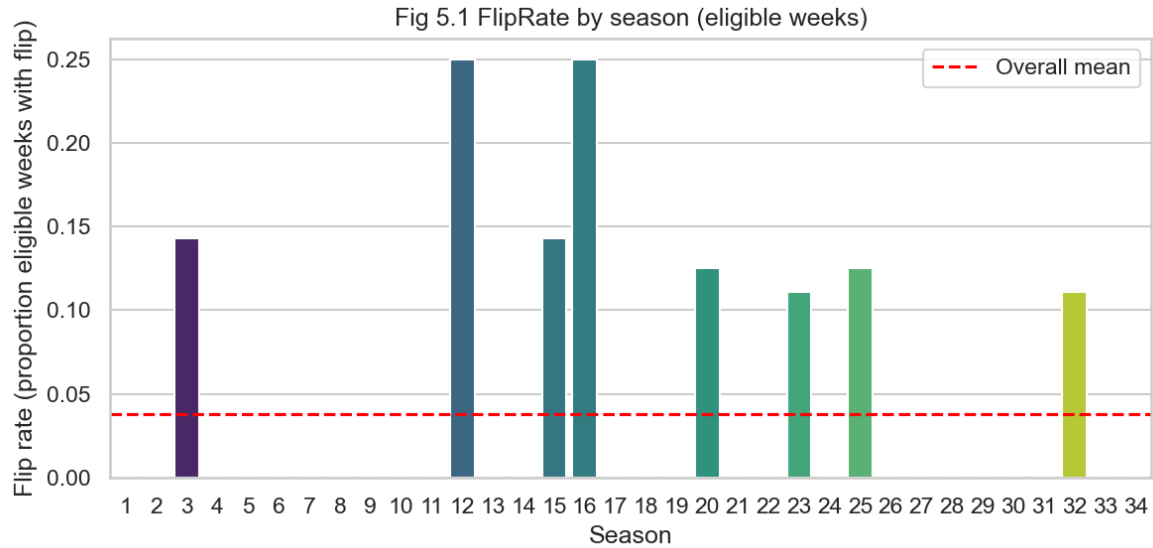


Figure 5: FlipRate by season under comparable (eligible) weeks.

5.1.1 Replay protocol and eligibility

Eligibility and special episodes. We treat eligibility as part of the replay interface: all week-level comparisons and KPI denominators are defined over the active set, and special episodes (no-elimination weeks, multi-elimination weeks, withdrawals, and finals) are handled explicitly rather than being forced into a single-elimination template. This design ensures that rule comparisons reflect the mechanics actually used on the show.

Aggregation rules (rank vs. percent). We replay eliminations under two official aggregation paradigms: rank-based and percent-based. For percent-based aggregation, the combined score is

$$S_{i,t}^{(P)}(\alpha) = \alpha q_{i,t}^J + (1 - \alpha) \hat{p}_{i,t},$$

and the replay eliminates the $m_{s,t}$ contestants with smallest $S_{i,t}^{(P)}(\alpha)$, where $m_{s,t}$ is the number of eliminations in week (s, t) . For rank-based aggregation, we replace the score shares by within-week ranks and apply the same elimination logic. Across both paradigms, ties are broken deterministically using a fixed secondary key (e.g., judges' score, then stable contestant ID), to preserve reproducibility.

5.1.2 Cross-season summary metrics

We compute replay-based summary metrics over counted (season, week) instances under a unified eligibility convention. KPI1 (Jaccard set agreement) summarizes week-level agreement between the observed eliminated set and the replayed eliminated set, including no-elimination weeks via an explicit empty-set convention. KPI3 (FlipRate) measures how often the predicted eliminated set differs between percent-based and rank-based replays under the same inputs.

Interpretation. High FlipRate indicates a season or era where the two aggregation paradigms can meaningfully change who is eliminated, which is a direct proxy for controversy risk when rules change. These metrics motivate the targeted case studies below, which explain *why* divergences occur.

5.2 Controversial Contestants: Mechanism-driven Case Studies

Aggregate metrics such as FlipRate identify *where* rules diverge, but they do not explain *why* a particular outcome was perceived as controversial. We therefore adopt mechanism-driven case studies: each selected contestant represents a distinct interaction between judges’ performance signals and audience support under the incumbent “combined Bottom-2 + judges’ save” pipeline and under alternative aggregation semantics (rank vs. percent).

Selection rationale (what makes a case “controversial” in our framework). A contestant is considered controversial if their survival path is supported by a persistent mismatch between judges’ signals and inferred fan support, so that small changes in aggregation (rank vs. percent) or stage-ordering (when judges intervene) can plausibly alter the elimination boundary. Operationally, we focus on seasons/episodes with elevated FlipRate and on contestants frequently cited in public discourse about judge–fan disagreement. We analyze four representative contestants: Jerry Rice (Season 2), Billy Ray Cyrus (Season 4), Bristol Palin (Season 11), and Bobby Bones (Season 27). All summaries are computed from `clean_long_data_new1.csv` using `season`, `week`, `celebrity_name`, `placement`, `elim_week`, and judges-derived variables (`J_total`, `J_pct`, `active`).

Protocol (kept fixed across cases). For each contestant we report: (i) their within-season judges trajectory (e.g., week-level position under $q_{i,t}^J$), (ii) their inferred fan-support trajectory through $\hat{p}_{i,t}$ (Task A), and (iii) counterfactual replays under percent vs. rank aggregation with and without the judges’ save module. The objective is not to “explain away” controversy, but to isolate the minimal mechanism that makes the outcome boundary-sensitive.

Case 1: Jerry Rice (Season 2) — early-era aggregation sensitivity. This case illustrates how early-season aggregation semantics can amplify judge–fan discordance: a contestant can remain viable when fan support offsets weak judges’ scores, yet the elimination boundary becomes sensitive to whether scores are combined as ranks or as normalized shares. In our replay framework, small changes in aggregation can change which contestants enter the Bottom-2 risk set, which then cascades through subsequent weeks.

Case 2: Billy Ray Cyrus (Season 4) — boundary effects without outcome reversal. Not every “controversial” public narrative implies a counterfactual outcome reversal. This case serves as a negative control: despite persistent discussion, our replays indicate that the elimination week can remain stable under the incumbent pipeline. We retain it to show that controversy perception can arise from judges–fan disagreement even when the realized elimination is not easily flipped by reasonable rule perturbations.

Case 3: Bristol Palin (Season 11) — popularity protection under fan-sensitive boundaries. This case represents a common mechanism in celebrity competitions: strong fan support can keep a weak performer out of the Bottom-2 set for multiple weeks, especially when the risk set is defined by a combined score. When a judges’ save is applied only after the Bottom-2 is formed, the mechanism protects technical merit only at the final step and cannot prevent fan support from shaping entry into the risk set.

Case 4: Bobby Bones (Season 27) — late-era mismatch and institutional interpretation. This case highlights that the same observed trajectory can be interpreted differently depending on institutional design. Under the incumbent “combined Bottom-2 + judges’ save” pipeline, fan support can strongly affect who enters the risk set; under a professionalism-first gate (Task D2), the risk set is defined by judges alone, which changes whether fan support can “rescue” a weak performer. We use this case to motivate why stage-ordering (gate vs. final save) is a central institutional lever rather than a cosmetic change.

Mechanisms synthesized across cases. Across the four examples we observe three recurring mechanisms: (i) **aggregation semantics** (rank vs. percent) can shift the boundary of who is at risk; (ii) **risk-set formation** is often more consequential than the final tie-break or save decision; and (iii) **stage-ordering** determines whether judges’ influence acts as a last-step safeguard (incumbent) or as a professionalism-first gate (Task D2). These mechanisms explain why controversies cluster around boundary-sensitive weeks and motivate the institutional redesign evaluated in Section 7.

5.3 Bottom-two and Judges’ Save Mechanism

Some later seasons introduce a two-stage mechanism: (i) identify a bottom subset using the vote aggregation rule, then (ii) select the eliminated contestant from that subset via judges’ save. To isolate the incremental effect of this change, we implement a modular judges’ save layer that can be toggled on top of either percent-based or rank-based aggregation.

Module definition. In a week with a bottom-two format, the replay first forms a bottom-two set $B_{s,t}$ using the chosen aggregation rule. The saved contestant is the one with higher judges’ score $J_{i,t}$ (equivalently higher $q_{i,t}^J$), and the other is eliminated. This construction cleanly separates *how the bottom set is formed* from *how the final elimination is chosen*, enabling controlled counterfactual comparisons.

Evaluation questions. We quantify (a) how often the judges’ save reverses the elimination implied by votes alone, (b) whether the save disproportionately protects contestants with strong judges’ scores but weak fan support, and (c) how the mechanism affects overall rule divergence and late-season stability. These results connect directly to management decisions about perceived fairness versus entertainment value.[2, 12]

Illustrative case-study replay. As a directional example, we run a Bottom-2 + judges’ save replay with $\alpha = 0.5$ (equal weight between $q_{i,t}^J$ and $\hat{p}_{i,t}$ in the combined score definition above) on Seasons 2/4/11/27. Under this module, contestants whose survival historically relies on fan support tend to be eliminated earlier: Jerry Rice (S2) is eliminated in Week 7, Bristol Palin (S11) in Week 9, and Bobby Bones (S27) in Week 8 in our replay, while Billy Ray Cyrus (S4) remains unchanged (Week 8).

Caveat. These outcomes can change for marginal weeks under different α values (we chose $\alpha \in [0.3, 0.7]$ in practice), tie-breaking conventions, and the semantics of multi-elimination weeks (we apply eliminations sequentially when multiple eliminations occur), so we treat the above as illustrative rather than universal.

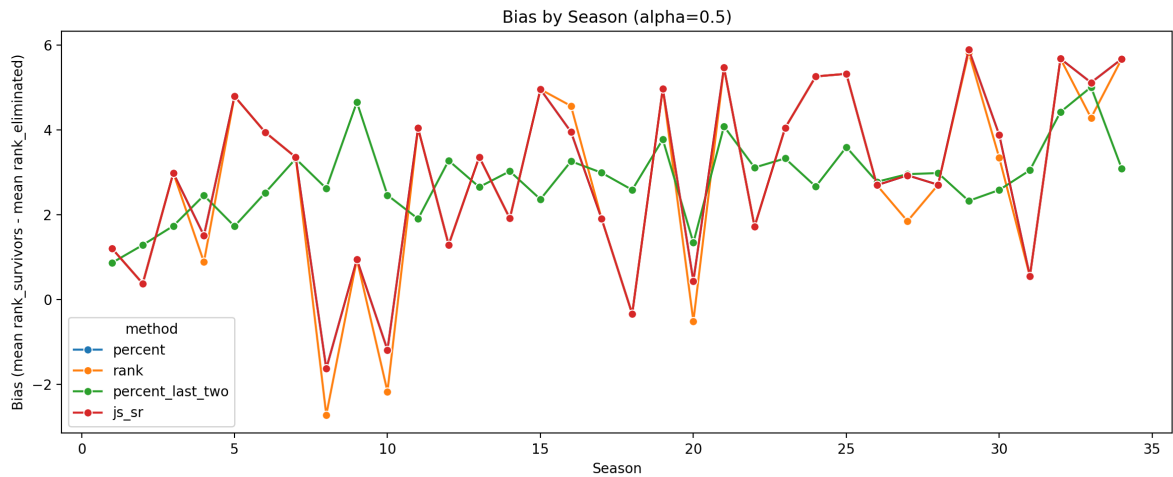


Figure 6: Average rank difference (bias) of eliminees vs. survivors across seasons by methods (percent/rank/Rule A 7.1.2/Rule B 7.1.3) at $\alpha = 0.5$.

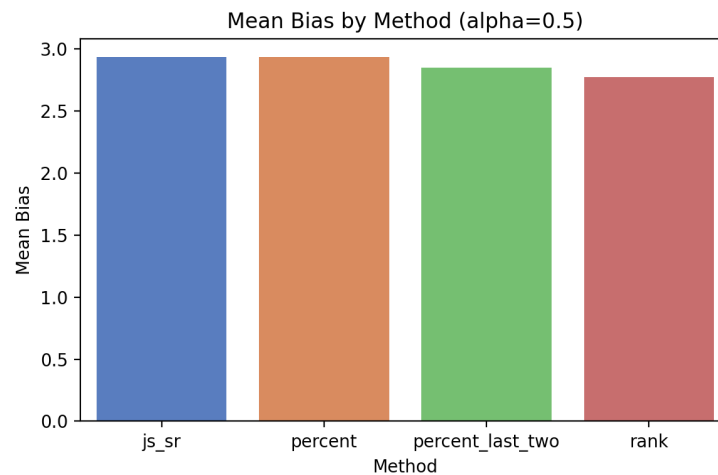


Figure 7: Method-level average summary of bias

5.4 Synthesis of Findings from Rules

We synthesize the rule replay evidence into actionable takeaways for both mechanism understanding and rule design. First, rank versus percent aggregation differences are not uniformly distributed across seasons; they are amplified in weeks where contestants cluster near the elimination boundary, making the system sensitive to small score-share changes. Second, the bottom-two + judges' save module acts as a structural preference for judges at the final decision margin, reducing the extent to which fan support can rescue a contestant once the contestant enters the bottom set. Third, these rule interactions motivate the professionalism-first institutional redesign in Task D-2: moving the judges' influence from a final-step safeguard to a gating mechanism, while preserving watchability through structured late-stage suspense mechanisms evaluated via replay and stress testing.

6 Task D-1: Impact Analysis: Pro Dancers & Celebrity Attributes

6.1 Outcome definitions

To disentangle how outcomes are shaped by professional partners versus celebrity attributes, we model judges' evaluations and inferred fan preferences as two distinct but related outcomes. For comparability across seasons and weeks, we work with *shares* rather than raw scores. Let $A_{s,t}$ denote the active set in season s , week t .

Judges' outcome. Define the judges' score share

$$q_{i,t}^J = \frac{J_{i,t}}{\sum_{r \in A_{s,t}} J_{r,t}}, \quad (15)$$

and use a logit transform with a small ϵ for stability:

$$y_{i,t}^J = \log \frac{q_{i,t}^J + \epsilon}{1 - q_{i,t}^J + \epsilon}. \quad (16)$$

Fans' outcome. Let Task A provide the inferred vote share $p_{i,t}$ (denoted $q_{i,t}^V$). We similarly define

$$y_{i,t}^V = \log \frac{p_{i,t} + \epsilon}{1 - p_{i,t} + \epsilon}. \quad (17)$$

We adopt share + logit for cross-season comparability and interpret coefficients on the log-odds scale.

Elimination outcome. Define a weekly elimination indicator

$$E_{i,t} = \begin{cases} 1, & \text{if contestant } i \text{ is eliminated in week } t, \\ 0, & \text{otherwise.} \end{cases}$$

This outcome links the two channels (judges' evaluations and inferred fan support) to realized competition survival.

6.2 Modeling results: separating “who judges reward” vs. “who fans support”

We fit two parallel mixed-effects models, one for judges and one for fans, with shared covariates but distinct random-effects structures. This allows us to quantify whether professional dancers act primarily as (i) performance amplifiers in the judges' channel, (ii) popularity amplifiers in the fans' channel, or both.

6.2.1 Model 1: Judges' evaluation model

We model judges' logit-share as

$$y_{i,t}^J = \beta_0 + X_i^\top \beta + f(t) + \text{FE}_s + v_{d(i)} + u_i + \varepsilon_{i,t}^J, \quad (18)$$

where X_i are celebrity attributes (e.g., age, industry), $f(t)$ captures week effects, FE_s are season fixed effects, $v_{d(i)}$ is a random intercept for the professional dancer partnered with celebrity i , and u_i is a contestant random effect capturing persistent idiosyncrasy. The variance component σ_v^2 represents the professional-dancer effect on judges' evaluations.

Hypothesis test. A key hypothesis test is $H_0 : \sigma_v^2 = 0$ (no pro-dancer effect on judges).

6.2.2 Model 2: Fans' preference model

We model fans' logit-share as

$$y_{i,t}^V = \gamma_0 + X_i^\top \gamma + f(t) + \text{FE}_s + b_{d(i)} + u_i + \varepsilon_{i,t}^V, \quad (19)$$

where $b_{d(i)}$ captures the professional-dancer effect on fan preferences. This is structurally parallel to Model 1, enabling a direct comparison of variance components and attribute coefficients.

Bridging judges to fans. To test whether judges' evaluations transmit to fan support, we also fit an augmented specification:

$$y_{i,t}^V = \gamma_0 + \gamma_J y_{i,t}^J + X_i^\top \gamma + f(t) + \text{FE}_s + b_{d(i)} + u_i + \varepsilon_{i,t}^V. \quad (20)$$

The coefficient γ_J measures the transmission strength from judges to fans.

6.2.3 Attribute preference gaps: judges vs. fans

To quantify whether fans and judges value celebrity attributes differently, we compare the attribute coefficients across the two channels. Let β_k and γ_k denote the coefficient on attribute k in Models 1 and 2. Define the preference gap

$$\Delta_k = \gamma_k - \beta_k. \quad (21)$$

$\Delta_k > 0$ indicates fans prefer the attribute more than judges do; $\Delta_k < 0$ indicates the opposite. We report Δ_k with uncertainty bands (via bootstrap) whenever vote inference uncertainty is non-negligible.

Model 3: Elimination risk (competition outcome). To connect both channels to realized outcomes, we fit a discrete-time survival model for weekly elimination:

$$\Pr(E_{i,t} = 1) = \text{logit}^{-1} \left(\theta_0 + \theta_J y_{i,t}^J + \theta_V y_{i,t}^V + \theta^\top X_i + f(t) + \text{FE}_s + r_{d(i)} \right),$$

where $E_{i,t}$ is the elimination indicator from Section 6.1, and $r_{d(i)}$ captures professional-dancer heterogeneity in outcome risk. This model quantifies how judges' evaluations, inferred fan support, and observed attributes jointly translate into elimination probability.

Quantifying professional-dancer impact (magnitude, not just significance). We quantify the fraction of outcome variance attributable to professional dancers using ICC-style ratios:

$$ICC_{pro}^J = \frac{\sigma_v^2}{\sigma_v^2 + \sigma_u^2 + \sigma^2}, \quad ICC_{pro}^V = \frac{\sigma_b^2}{\sigma_b^2 + \sigma_u^2 + \sigma^2}, \quad (22)$$

where σ_v^2 is the variance of $v_{d(i)}$, σ_b^2 is the variance of $b_{d(i)}$, σ_u^2 is the contestant-level variance, and σ^2 is the residual variance. These ratios provide an interpretable measure of how much professional partners matter relative to persistent celebrity idiosyncrasy.

6.2.4 Uncertainty propagation from vote inference

Because $p_{i,t}$ is inferred rather than observed, we propagate vote-inference uncertainty into downstream coefficients. We do so either by (i) bootstrap resampling of the inference stage and re-fitting Models 2–3, or (ii) weighted regression using the interval width from Task A as an uncertainty proxy; we report confidence intervals accordingly. In the weighted specification, let $w_{i,t}$ denote the feasible vote-interval width from Task

A (KPI2). We set observation weights

$$\omega_{i,t} = \frac{1}{w_{i,t} + \varepsilon},$$

so that more identifiable (tighter-interval) weeks exert greater influence, while poorly identified weeks are down-weighted rather than discarded.

7 Task D-2: New Rule Design and Stress Testing

7.1 Task D-2: New Rule Design and Stress Testing

7.1.1 Design objective: professionalism-first, watchability as a constrained secondary goal

Because DWTS is fundamentally a performance competition, we adopt a *professionalism-first* rule design principle: eliminations should be predominantly driven by judges' performance signals. At the same time, DWTS is a televised product, so the system must preserve watchability by keeping a meaningful audience role and maintaining suspense. Importantly, any additional uncertainty must remain *performance-grounded* (induced by explicit rule structure operating on low performers), rather than arbitrary randomness.

Operational definitions. We define **professionalism** as the extent to which eliminated contestants are concentrated in the lower tail of the weekly judges' ranking. We define **watchability** as a constrained secondary objective that requires (i) a non-trivial marginal effect of fan votes on survival (audience influence) and (ii) suspense that avoids overly deterministic outcomes, while remaining performance-grounded.

Notation. For each season-week (s, t) , let $q_{i,t}^J$ denote contestant i 's normalized judges' score share within the active set, and let $p_{i,t}$ denote the inferred/proxy fan vote share from Task A. For a weight parameter $w_J \in [0, 1]$, define a combined score

$$S_{i,t} = w_J q_{i,t}^J + (1 - w_J) p_{i,t}.$$

7.1.2 Bottom-2 Rule (B2 Rule): combined Bottom-2 $\rightarrow$ judges' save

Bottom-2 (B2) rule represents the show's baseline mechanism in which judges' scores and fan votes are first combined into a weekly ordering, after which the final elimination is decided by judges within the Bottom-2 risk set (a "judges' save" safeguard).

1. **Combined ranking.** For each active contestant i , compute the combined score

$$S_{i,t} = w_J q_{i,t}^J + (1 - w_J) p_{i,t}$$

and form the combined ranking for week t .

2. **Risk set (Bottom-2).** Let B_t be the set of the two lowest-ranked contestants under $S_{i,t}$.
3. **Judges' save.** Within B_t , compare judges' performance and eliminate the contestant with the lower judges' signal:

$$\tilde{E}_{s,t} = \arg \min_{i \in B_t} q_{i,t}^J.$$

Equivalently, the contestant with the higher judges' score within the Bottom-2 is saved.

Interpretation. preserves audience influence because fan support affects who enters the Bottom-2, while the judges' save prevents extreme popularity from overriding performance at the final decision margin. However, because the risk set itself is defined by the combined score, strong fan support can still keep a weak

performer out of the Bottom-2, which limits professionalism—especially in early weeks when performance dispersion is large.

7.1.3 *Professional Bottom-2 Rule (PB2): judges-only gating* → fan-aware decision within the risk set

Compared to *B2*, our key modification is to move the judges-only safeguard from the decision stage to the gating stage. Our proposed institutional change is a *two-stage gate-and-decide* mechanism that makes professionalism structural: judges alone define who is at risk, and fan influence is applied only within that performance-defined boundary.

1. **Judges-only gate (risk set).** Rank contestants by judges' performance $q_{i,t}^J$ and let B_t^J be the Bottom-2 under judges (Judge Bottom-2).
2. **Fan-aware decision within the gate.** Within B_t^J , compute $S_{i,t}$ and eliminate the contestant with the lower combined score:

$$\tilde{E}_{s,t} = \arg \min_{i \in B_t^J} (w_J q_{i,t}^J + (1 - w_J) p_{i,t}).$$

Why *Professional Bottom-2* improves professionalism. By construction, elimination is always decided among the lowest-performing contestants according to judges. This prevents extreme popularity from overriding performance signals outside the judges-defined bottom tail, thereby strengthening professionalism, especially in early weeks when performance dispersion is largest.

Maintaining watchability without randomness: phase-dependent structured suspense. Because DWTS features celebrity participants, early-season skill gaps can be large and a strict professionalism-first gate may reduce perceived audience impact. We therefore treat watchability as a constrained secondary goal and introduce *structured suspense* only in late stages, while maintaining performance grounding:

- **Late-stage bottom- k band (within judges' tail).** When the field is small (e.g., remaining contestants $\leq K$), expand the judges-defined risk set from Bottom-2 to Bottom- k , and decide elimination within that band using $S_{i,t}$. This widens the decision boundary and increases suspense without leaving the low-performance region.
- **Limited structured multi-elimination (optional).** In a small number of late-stage weeks, allow multi-elimination, but restrict eliminations to the judges-defined bottom band. This increases stakes while remaining performance-grounded.

The phase boundary (K or a week threshold t_0) and the band size k are interpretable management knobs.

7.1.4 Stress testing via replay

We evaluate Rule B relative to the incumbent Rule A through counterfactual replay on the canonical season-week-contestant panel, under the same eligibility conventions as earlier sections. The goal is not a single-point “optimal” parameter, but a transparent parameter region that is robust across season formats and vote-uncertainty scenarios.

Parameter sweep (control knobs). We sweep: (i) judge weight w_J , (ii) the phase boundary (K or t_0) for enabling late-stage mechanisms, and (iii) the late-stage band size k and the frequency/placement of structured multi-elimination weeks (if enabled).

Evaluation metrics (professionalism-first). We report:

- **Professionalism proxy: judges-tail alignment.** The eliminated contestant’s weekly judges’ rank/quantile distribution (greater concentration in the lower tail indicates higher professionalism).
- **Replay consistency (KPI1).** Week-level elimination-set agreement between observed and replayed outcomes (Jaccard mean under our stated convention).
- **Structured suspense proxy (KPI3 / FlipRate).** The rate at which elimination outcomes change across rules/parameters, capturing how institutional design introduces suspense.
- **Audience influence proxy.** Sensitivity of elimination outcomes to perturbations of $p_{i,t}$ within the feasible uncertainty range from Task A.

Robustness checks. We conduct three robustness checks:

1. **Vote-uncertainty perturbations:** perturb/resample $p_{i,t}$ and track elimination and winner stability.
2. **Phase-boundary sensitivity:** vary K (or t_0) and late-stage strength (k and multi-elimination placement) to test brittleness.
3. **Format variation:** include special episodes (no-elimination and multi-elimination weeks) and verify decisions remain performance-grounded.

In summary, Rule B makes professionalism a structural property through judges-only gating, while late-stage structured mechanisms preserve watchability without introducing randomness. Replay-based stress testing provides an interpretable policy region rather than a single tuned setting.

8 Conclusion

In this study, we model *Dancing with the Stars* as an inverse problem in which judges’ scores and eliminations are observed while audience vote totals are withheld. Task A provides a convex inference framework to reconstruct weekly fan vote shares consistent with observed eliminations and to quantify identifiability and uncertainty. Building on these inferred votes, Tasks B and C implement season–week replays that isolate the impact of aggregation mechanics (percent versus rank) and the incumbent “combined Bottom-2 + judges’ save” safeguard, revealing where rule changes most strongly affect outcomes. When a rule change keeps an observed-exit contestant active, we extend the replay using a conservative, minimal-information imputation solely for counterfactual completion, not as a performance forecast. Task D1 then separates the drivers of judges’ evaluations and fan support using mixed-effects structures that attribute variation to celebrity attributes and professional partners.

Finally, Task D2 proposes a professionalism-first institutional redesign based on stage ordering: judges alone define a small risk set (gate), and fan influence is applied only within that performance-defined boundary via a transparent weighted blend. To preserve watchability without introducing randomness, we introduce late-stage structured, performance-grounded suspense mechanisms (e.g., expanding the judges-defined risk set to a bottom- k band or using limited multi-elimination), and validate the resulting policy region through replay-based parameter sweeps and vote-uncertainty stress tests.

Collectively, our pipeline produces an auditable chain from data to mechanism interpretation to actionable rule recommendations, while explicitly flagging weeks and seasons where non-identifiability limits the strength of conclusions. Future work could incorporate richer behavioral signals (e.g., external popularity proxies) and explore alternative mechanism designs under the same replay-and-stress-test framework.

9 Memo to Management

To: Show Management

Subject: Recommendation for a More Professional Yet Watchable Voting System

Our analyses indicate that controversy risk concentrates in weeks where judges' evaluations and audience preferences diverge. To prioritize technical merit while preserving an audience role, we recommend piloting a *Judges-Gated Two-Stage* (JG-2S) voting policy, validated by historical replays and uncertainty stress tests.

Baseline reference (current mechanism). The incumbent format can be summarized as “combined Bottom-2 + judges’ save”: contestants are first ranked by a combined judge–fan score, the Bottom-2 form an at-risk set, and judges decide which of the two is eliminated.

Recommended policy (JG-2S): judges-only gate → fan-aware decision within the gate. Under JG-2S, each week proceeds in two stages:

1. **Judges-only gate (risk set).** Rank contestants by judges' performance share $q_{i,t}^J$ and designate the Bottom-2 (or Bottom- k in late-stage variants) as the at-risk set.
2. **Fan-aware decision within the gate.** Within the at-risk set, eliminate the contestant with the lower transparent blended score

$$S_{i,t} = w_J q_{i,t}^J + (1 - w_J) p_{i,t},$$

where $p_{i,t}$ denotes the inferred/proxy fan vote share.

Why this improves professionalism. Because elimination is always decided among the judges-defined bottom performers, extreme popularity cannot keep a weak performer out of the at-risk set. This makes professionalism a structural property of the system rather than a tunable side effect.

How we preserve watchability (without randomness). To maintain suspense while remaining performance-grounded, we recommend a phase-dependent schedule: early weeks use Bottom-2 gating to emphasize professionalism; late in the season (when remaining contestants are few and performance gaps narrow), management may enable structured suspense by expanding the gate to a Bottom- k band and/or using a small number of structured multi-elimination weeks, with eliminations restricted to the judges-defined low-performance band.

Management knobs and deployment. JG-2S offers interpretable control knobs: (i) w_J (judge–fan balance within the gate), (ii) the late-stage trigger (e.g., remaining contestants $\leq K$ or week threshold t_0), and (iii) the late-stage gate size k (and optional multi-elimination placement). We recommend selecting a robust parameter region identified by replay and vote-uncertainty stress tests, and deploying JG-2S in a one-season *shadow mode* computed in parallel with the broadcast rule. This yields an auditable comparison of eliminations, finalists, and winner outcomes before any on-air adoption, supported by a clear public explanation of the two-stage logic and the role of audience influence.

References

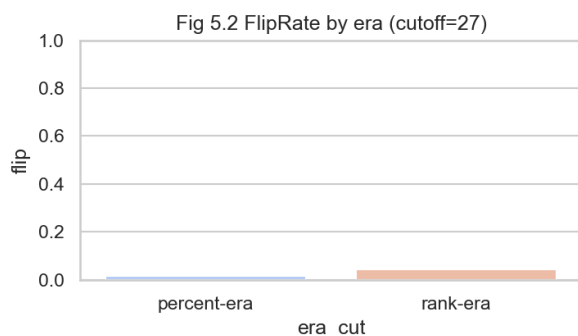
- [1] COMAP, “2026 MCM problem C dataset (problem_c_data.csv),” MCM/ICM official contest materials, 2026, accessed: 2026-01-31. [Online]. Available: https://www.contest.comap.com/undergraduate/contests/mcm/contests/2026/problems/2026_MCM_Problem_C_Data.csv
- [2] D. G. Saari, *Decisions and Elections: Explaining the Unexpected*. Cambridge University Press, 2001. [Online]. Available: <https://www.cambridge.org/core/books/decisions-and-elections/>
- [3] COMAP, “2026 MCM problem C: Data with the stars (problem statement pdf),” MCM/ICM official contest materials, 2026, accessed: 2026-01-31. [Online]. Available: https://www.contest.comap.com/undergraduate/contests/mcm/contests/2026/problems/2026_MCM_Problem_C.pdf

- [4] S. Bubeck, “Convex optimization: Algorithms and complexity,” *Foundations and Trends in Machine Learning*, vol. 8, no. 3-4, pp. 231–357, 2015. [Online]. Available: <https://www.nowpublishers.com/article/Details/MAL-050>
- [5] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004. [Online]. Available: <https://web.stanford.edu/~boyd/cvxbook/>
- [6] M. Benning and M. Burger, *Modern Regularization Methods for Inverse Problems*. Springer, 2018. [Online]. Available: <https://link.springer.com/book/10.1007/978-3-319-93978-0>
- [7] J. Kaipio and E. Somersalo, *Statistical and Computational Inverse Problems*. Springer, 2005. [Online]. Available: <https://link.springer.com/book/10.1007/b138659>
- [8] J. Aitchison, *The Statistical Analysis of Compositional Data*. Chapman and Hall, 1986.
- [9] V. Pawlowsky-Glahn and A. Buccianti, Eds., *Compositional Data Analysis: Theory and Applications*. Wiley, 2011. [Online]. Available: <https://onlinelibrary.wiley.com/doi/book/10.1002/9781119976462>
- [10] P. Filzmoser, K. Hron, and C. Reimann, “Compositional data analysis in practice,” in *Statistical Analysis of Compositional Data in the Geosciences*. Springer, 2018. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-96427-0_2
- [11] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, “Distributed optimization and statistical learning via the alternating direction method of multipliers,” *Foundations and Trends in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011. [Online]. Available: https://web.stanford.edu/~boyd/papers/admm_distr_stats.html
- [12] F. Brandt, V. Conitzer, U. Endriss, J. Lang, and A. D. Procaccia, Eds., *Handbook of Computational Social Choice*. Cambridge University Press, 2016. [Online]. Available: <https://www.cambridge.org/core/books/handbook-of-computational-social-choice/>

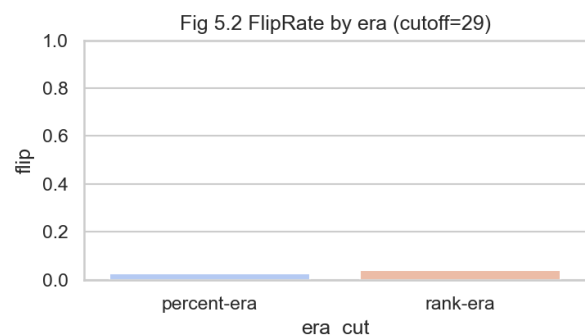
A Appendix: Data, Algorithms, and Reproducibility

Appendix A1: Cutoff Sensitivity Analysis

In this appendix, we complete the cutoff sensitivity analysis for our rule-replay module discussed in Section 7.1.4.



(a) Cutoff at Season-27.



(b) Cutoff at Season-29.

B Complete Files for Problem B

See [2625838](#) for repository tree and execution steps.

Report on Use of AI

Team Control Number: 2625838

Problem: C

AI Tools Used

AI Tool	Version / Model	Primary Purpose
OpenAI ChatGPT	GPT-5.2	Language polishing, structural refinement, and clarification of written explanations
OpenAI ChatGPT	GPT-5.2	Brainstorming ideas for modeling approaches, algorithm designs, and solution strategies
OpenAI ChatGPT	GPT-5.2	Generating preliminary code drafts for data processing, graph producing and simulation, later reviewed and modified by the team
OpenAI ChatGPT	GPT-5.2	Synthesis of the explanatory documents, work orders, template tickets for internal team Delivery
OpenAI ChatGPT	GPT-5.2	Giving Advice on engineered Document Management and Version Control best practices
Github Copilot	—	Enhance productivity while maintaining full control over the code quality and integrity

Query/Response Record

Click **ChatGPT Query #** to view the full input and output records on GitHub. Screenshots of the outputs are provided below for reference. [ChatGPT Query 1:](#)

Input: We just inished our setting of *dynamic_files*, please refer to the *first_stage_modeling.md* and *context_pack_for_ai.md* in our repository, and search for authoritative and highly credible supporting literature on topics of convex optimization/scalable solving, underdetermined inversion/regularization, ranking/rule differences, compositional data, simulation and sensitivity

Output:

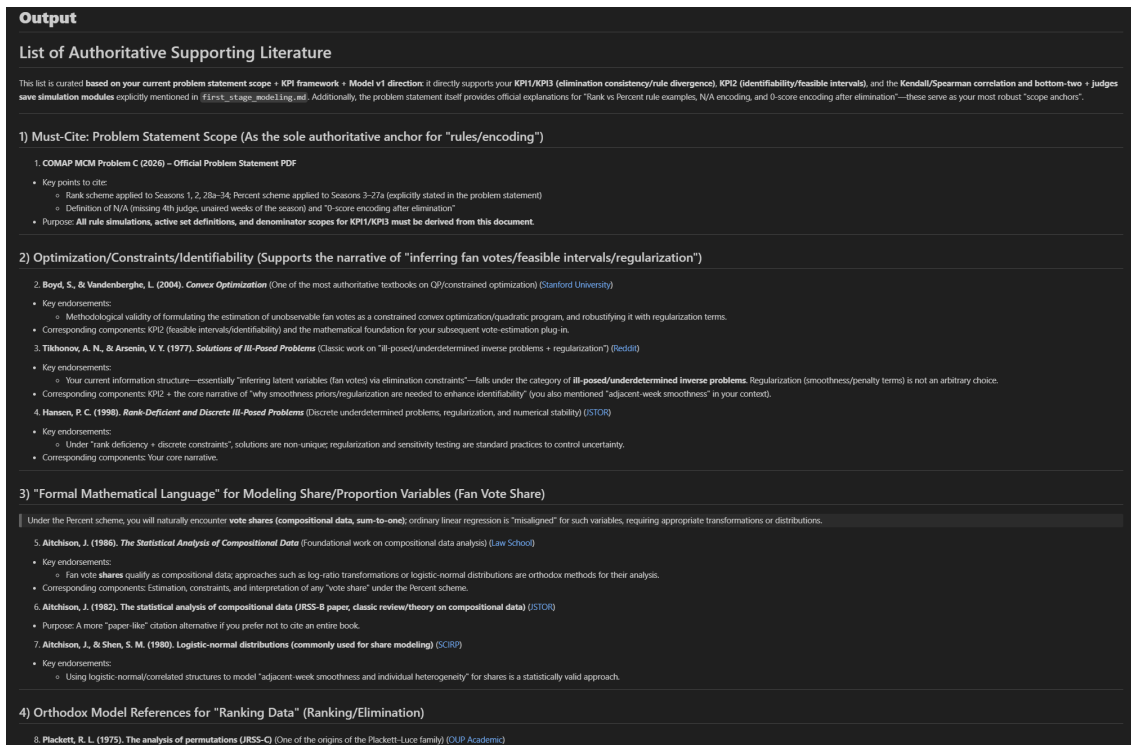


Figure 9: ChatGPT Query 1 Output Screenshot

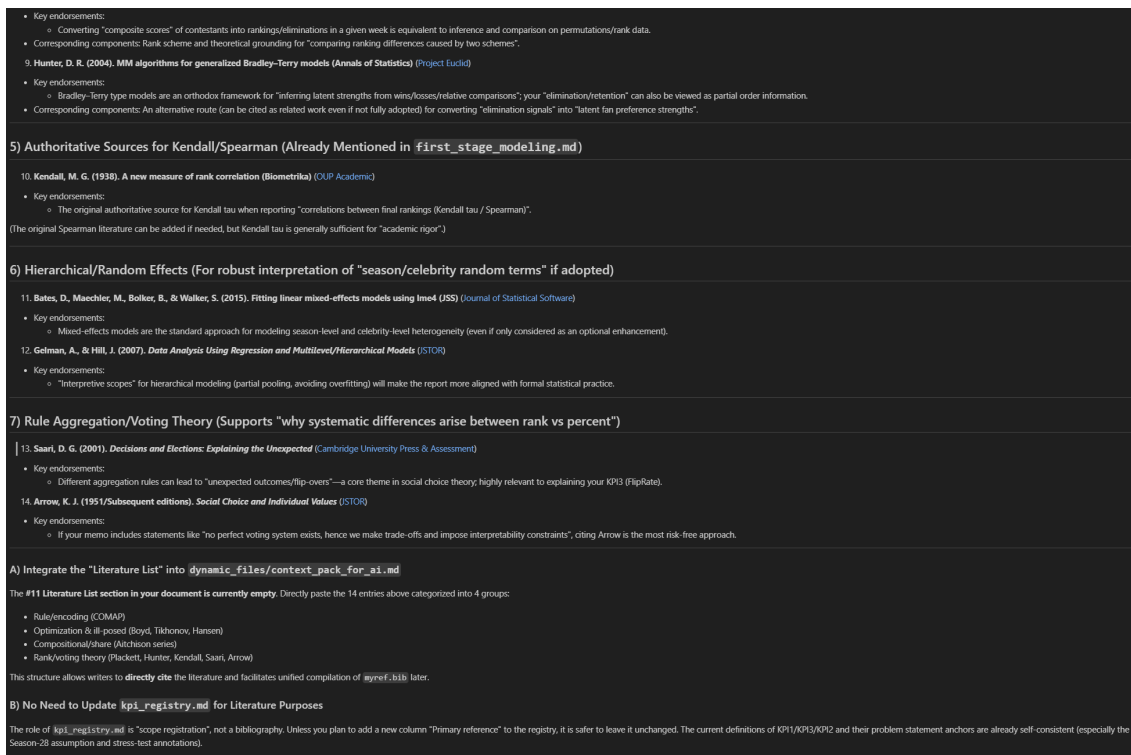


Figure 10: ChatGPT Query 1 Output Screenshot (Continued)

ChatGPT Query 2:

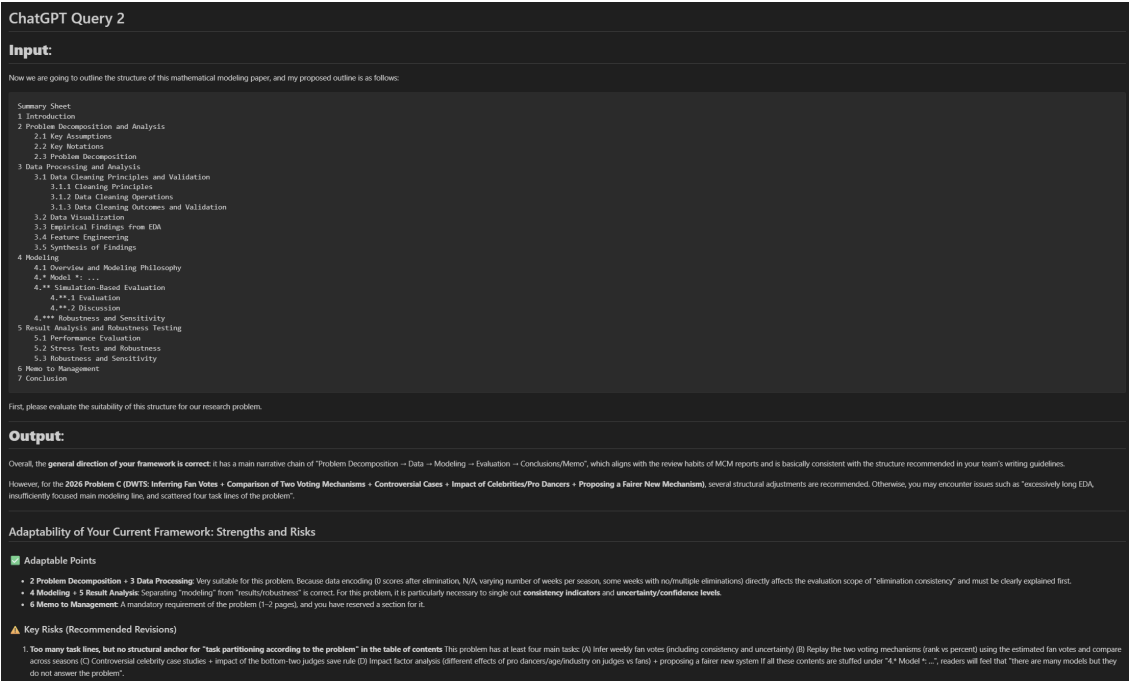


Figure 11: ChatGPT Query 2 Output Screenshot

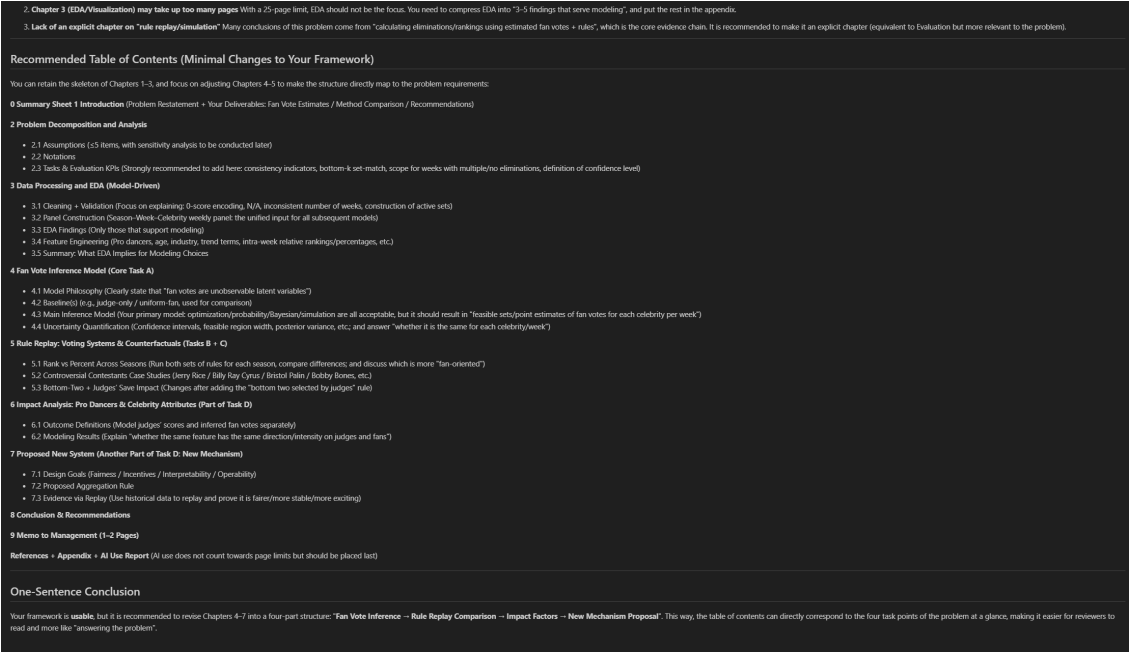


Figure 12: ChatGPT Query 2 Output Screenshot (Continued)

ChatGPT Query 3:
Input:

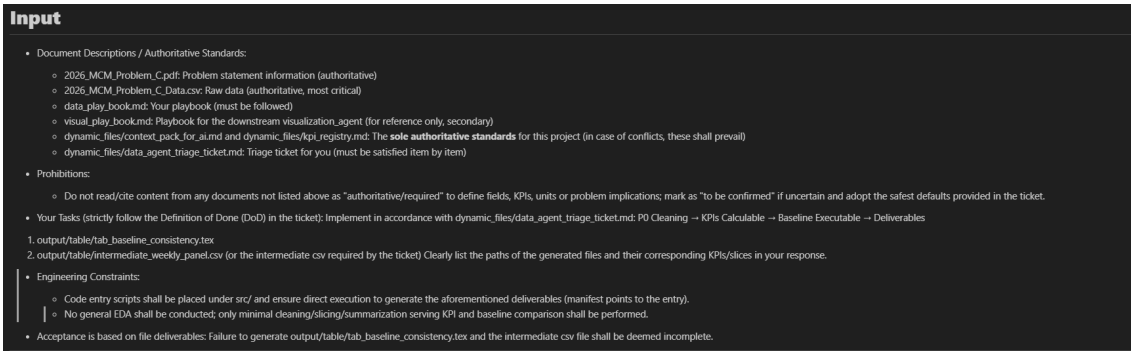


Figure 13: ChatGPT Query 3 Input Screenshot

Output:

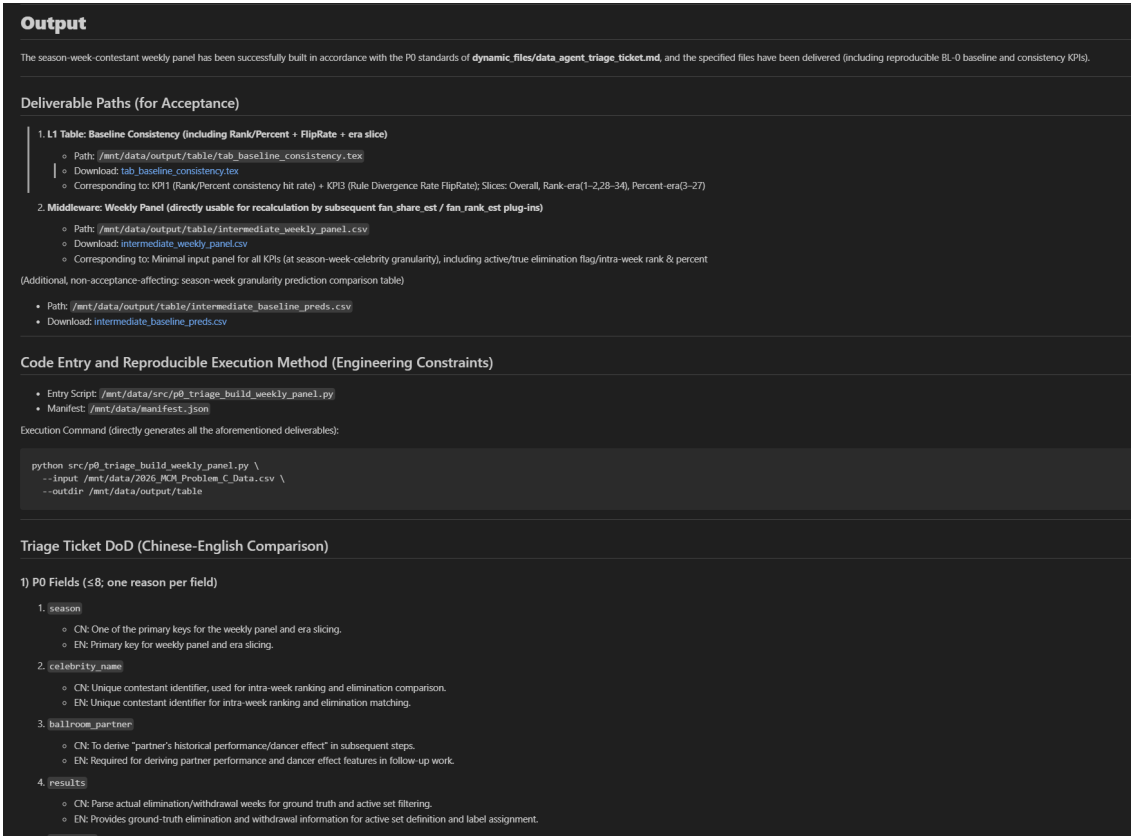


Figure 14: ChatGPT Query 3 Output Screenshot

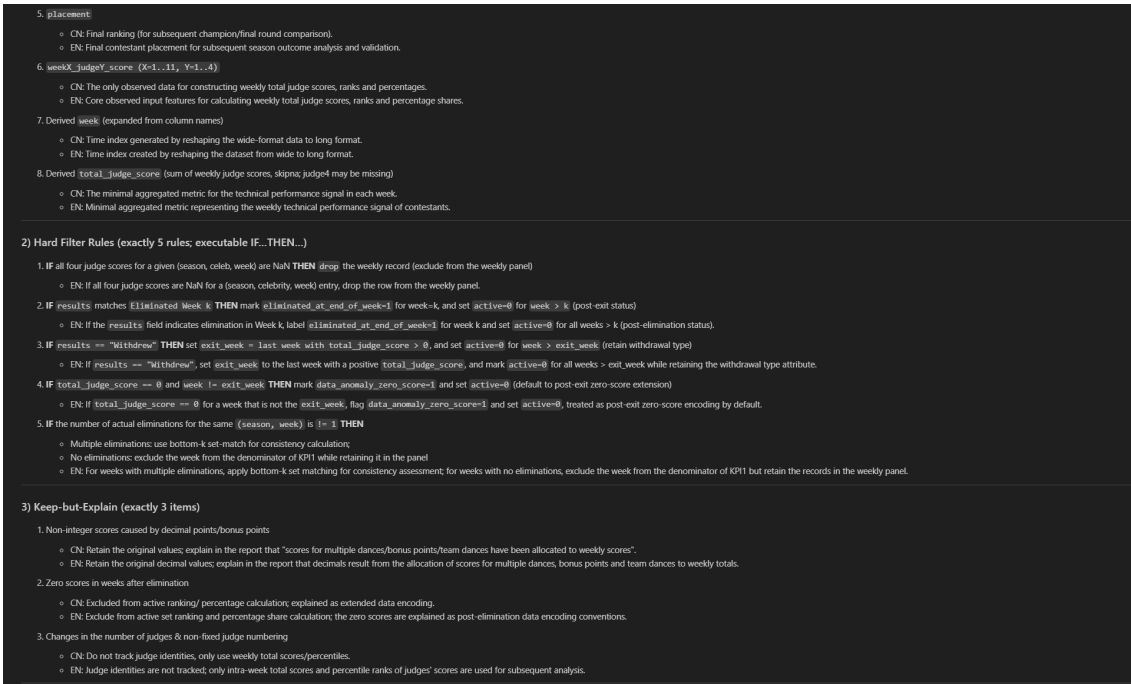


Figure 15: ChatGPT Query 3 Output Screenshot (Continued)

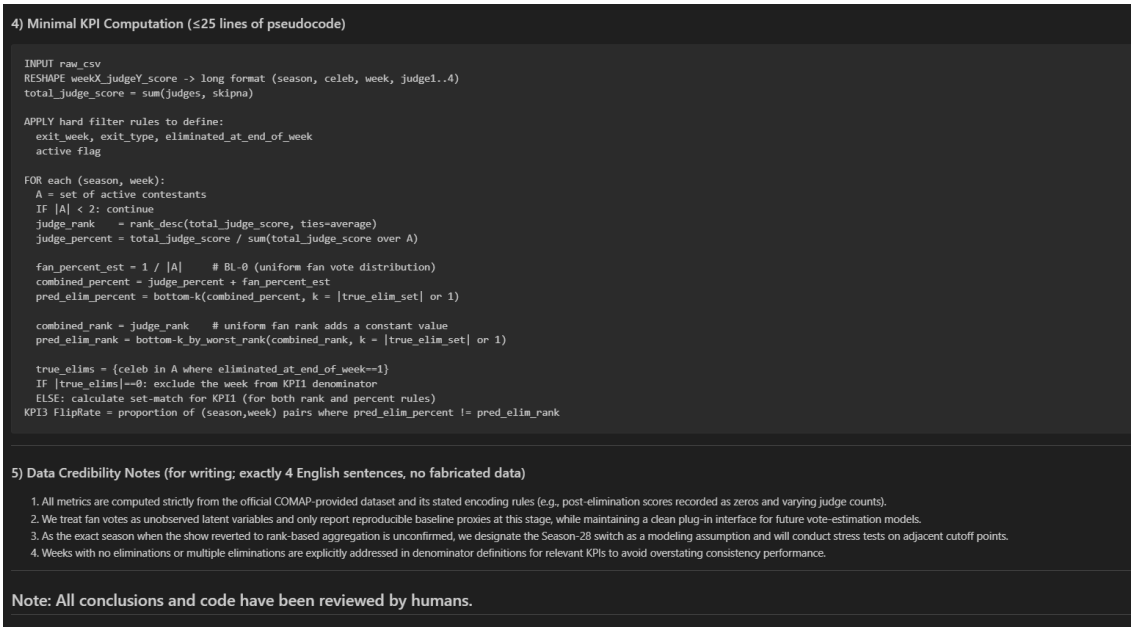


Figure 16: ChatGPT Query 3 Output Screenshot (Continued)

ChatGPT Query 4:

Input: Wait, why is the generated .pdf file completely unchanged? Could there be an issue with how we implemented the rules? Also, where in the code you provided do we need to modify the paths for importing and exporting files?

Output:

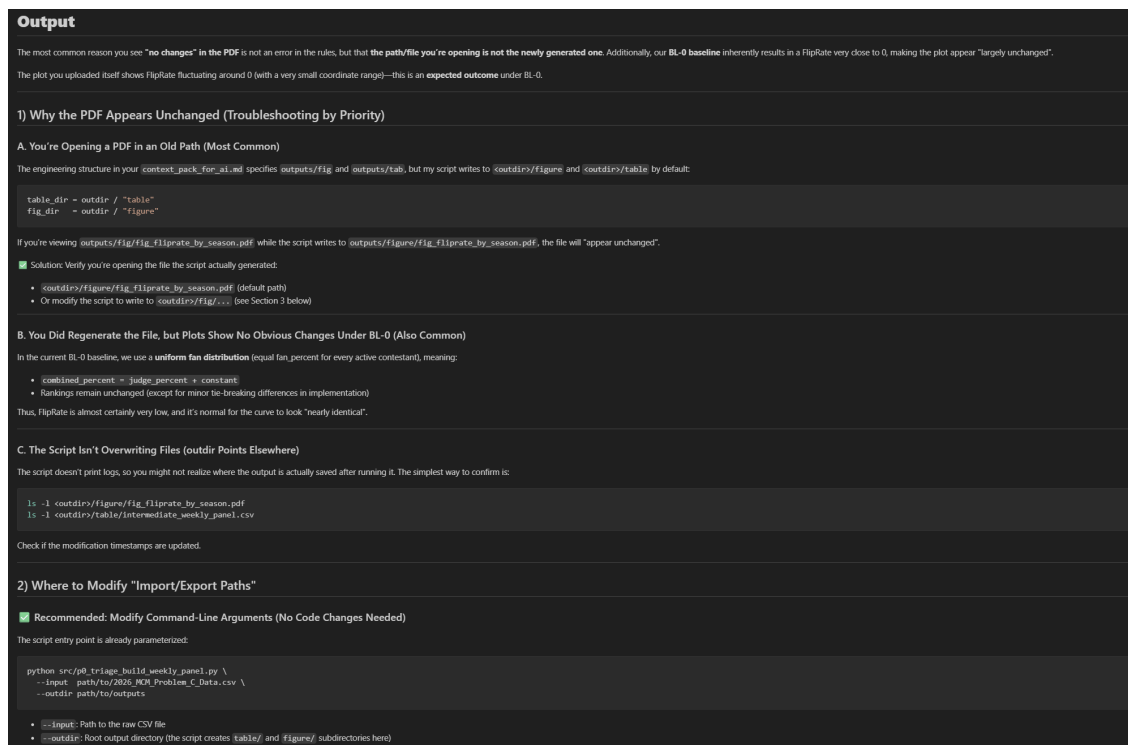


Figure 17: ChatGPT Query 4 Output Screenshot

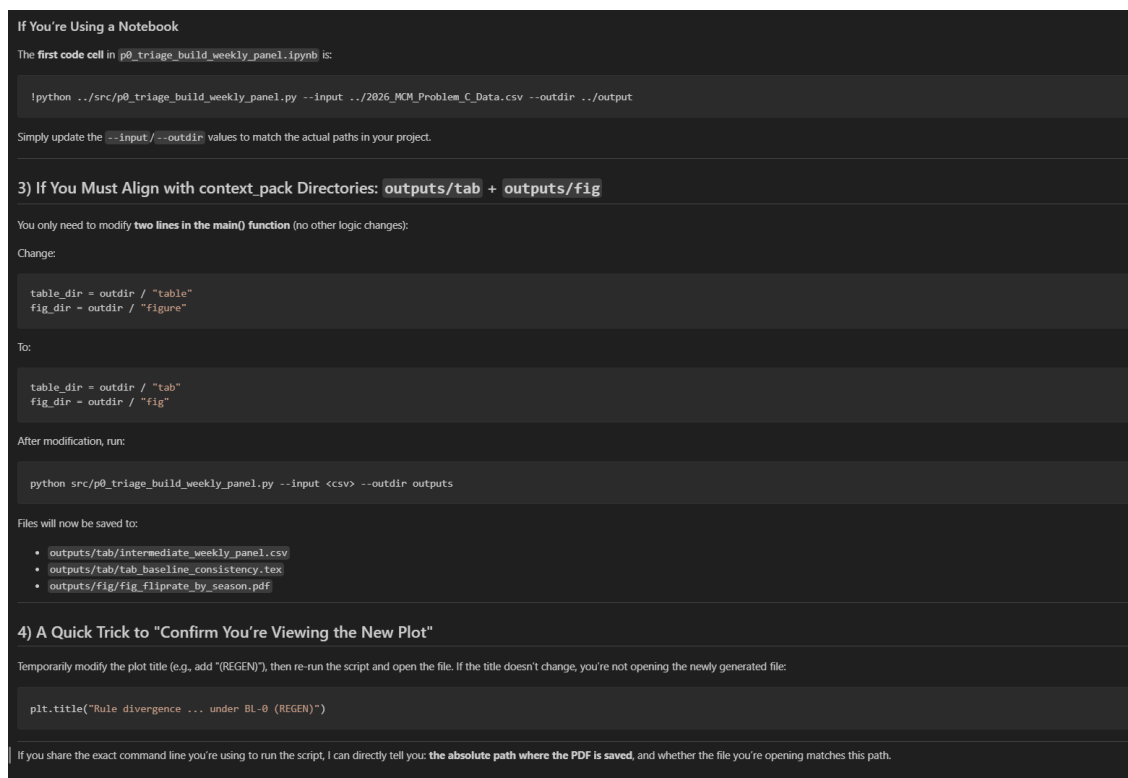


Figure 18: ChatGPT Query 4 Output Screenshot

ChatGPT Query 5:

Input: Currently, our .py code files are located at `2026_MCM/src/LI_play_data/*.py`, and we expect the generated charts to be saved to `2026_MCM/output/fig` (or `2026_MCM/output/tab` for tables, and the required data to be saved to `2026_MCM/output/data_cleaned`).

I have several questions:

- Do we need to manually create the output/fig and output/tab directories in advance?
- If not, do we need to modify the paths in the Python files, and how? (We do not plan to use .ipynb files.)
- After modifying the paths in the files, how should I run the script on a Windows computer to generate the new files? Will the newly generated files overwrite the existing ones after running the script?

Output:

Output

Understood: You currently place the entry script at `2026_MCM/src/LI_play_data/*.py` and expect outputs to be saved to:

- `2026_MCM/output/fig`
- `2026_MCM/output/tab`
- `2026_MCM/output/data_cleaned`

Below are step-by-step answers to your 4 questions, along with a **minimal modification approach** (only adjust paths/directory creation without changing core logic).

1) Do we need to manually create output/fig and output/tab in advance?

No (recommended). It is more reliable to let the script automatically create directories using `mkdir(parents=True, exist_ok=True)` (pathlib) or `os.makedirs(..., exist_ok=True)` (os module).
You only need to ensure the parent directory of `2026_MCM/output` exists (this is typically already present in the repository).

2) Do we need to modify paths in the .py files? How (without using ipynb)?

☒ **Recommended:** Keep the `--outdir` parameter, only modify "subdirectory name mapping"

Since you want outputs in `output/fig`, `output/tab`, and `output/data_cleaned`, modify the following lines in the script (usually in `main()` or before writing outputs):

Original code:

```
table_dir = outdir / "table"
fig_dir = outdir / "figure"
```

Modified code:

```
tab_dir = outdir / "tab"
fig_dir = outdir / "fig"
data_dir = outdir / "data_cleaned"

# Automatically create directories (no error if already exists)
tab_dir.mkdir(parents=True, exist_ok=True)
fig_dir.mkdir(parents=True, exist_ok=True)
data_dir.mkdir(parents=True, exist_ok=True)
```

Then replace all file-writing paths accordingly:

- LaTeX tables: Save to `tab_dir / "tab_baseline_consistency.tex"`
- Intermediate panel CSV: Save to `data_dir / "intermediate_weekly_panel.csv"` (as you specified "data to be saved in data_cleaned")
- Baseline predictions: Also save to `data_dir` (optional)
- PDF charts: Save to `fig_dir / "fig_figurate_by_season.pdf"`

Note: If your original code wrote to `<outdir>/table/...`, update it to `<outdir>/tab/...`; similarly, `<outdir>/figure/...` → `<outdir>/fig/...`

Another common issue: Default relative paths may fail due to script location changes

Your script is in `src/LI_play_data/`. If it uses relative paths (e.g., `../2026_MCM_Problem_C_Data.csv`), it is recommended to **avoid hardcoding** and continue passing the input path via the `--input` command-line argument.

3) How to run the script on Windows to generate new files?

Since you do not plan to use notebooks, run the script via the command line.

Method A: Run from the repository root directory (the level containing 2026_MCM/)

Works for both PowerShell and CMD. Example (PowerShell):

```
cd <your-repository-root-directory>
python .\2026_MCM\src\LI_play_data\p0_triage_build_weekly_panel.py `
  --input .\2026_MCM\2026_MCM_Problem_C_Data.csv `
  --outdir .\2026_MCM\output
```

CMD syntax (no backticks; use `⏎` for line breaks or write as one line):

Figure 19: ChatGPT Query 5 Output Screenshot

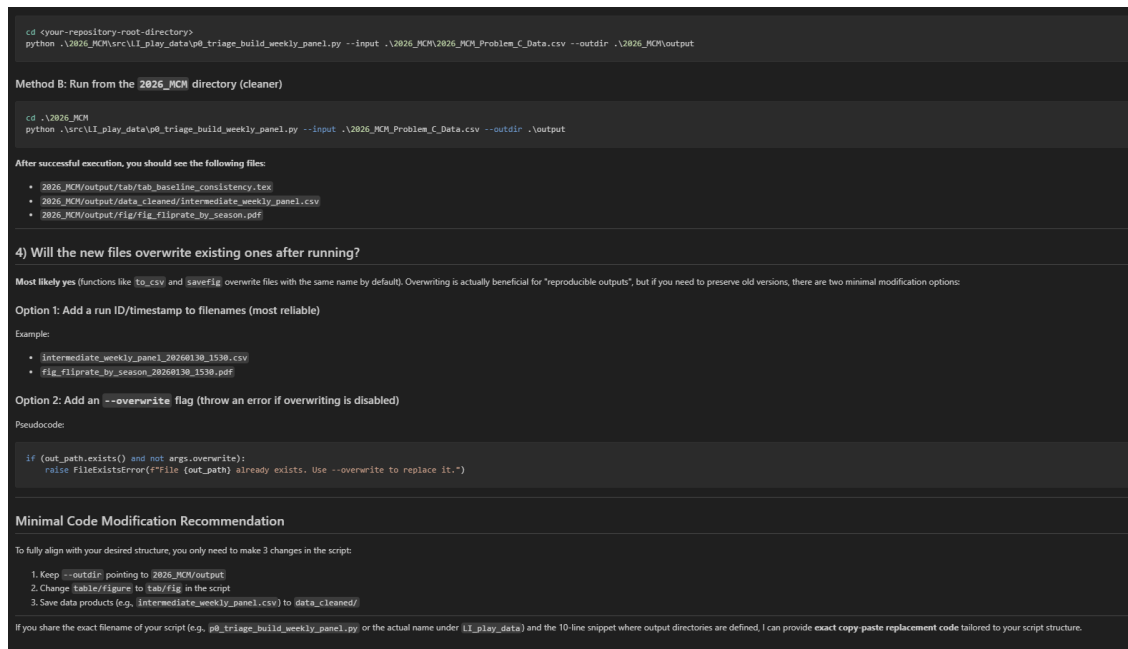


Figure 20: ChatGPT Query 5 Output Screenshot

ChatGPT Query 6:

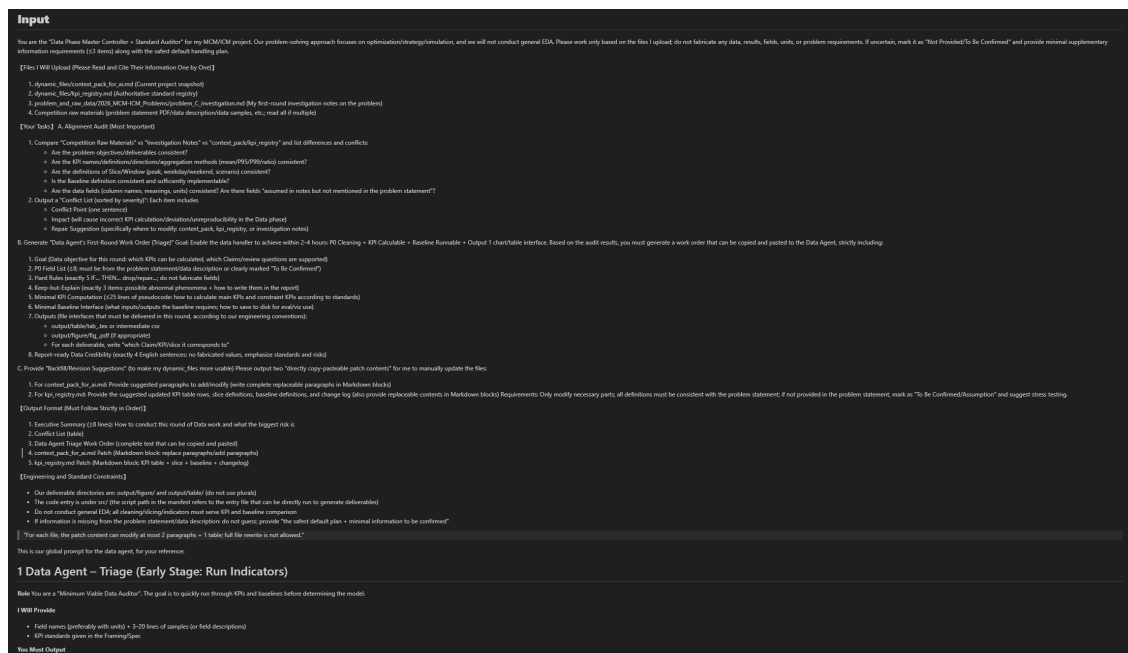


Figure 21: ChatGPT Query 6 Output Screenshot



```
combined_rank = judge_rank # Uniform fan rank adds a constant, omitted
pred_elim_rank = np.nan * total_rank

true_elim = (c in A with eliminated_at_end_of_season=1)
update counters for:
    kpi_elim_active_percent += not_match(pred_elim_percent, true_elim)
    kpi_elim_active_rank += not_match(pred_elim_rank, true_elim)

OUTPUT KPIs:
Elimination_percent = mean over eligible (season,week)
Elimination_rank = mean over eligible (season,week)
Eliminate = share of weeks where pred_elim_percent is pred_elim_rank

6) Minimal Baseline Interface (Inputs/Outputs Required for Baseline; How to Save to Disk)
- Input: weekly_game.csv (each row = season-week-club, including total_judge_score, judge_rank, judge_percent, active, true_elim_rank)
- Baseline Output:
    1. kpi_elim_active_percent / kpi_elim_active_rank (one predicted elimination or not per season-week)
    2. Elimination_percent / Combined_elim_percent (for eval/ve comparison)
- Save to Disk:
    1. kpi_elim_active_percent / kpi_elim_active_rank (shared for evaluation/development for vote models)
    2. kpi_elim_active_percent / kpi_elim_active_rank (shared for evaluation/development for vote models)

7) Outputs (Must Deliver Interfaces; Mark Corresponding Claims/KPI/Slice)
1. Update/Refresh/Intermediate/Consistency/Run (1 Table)
    - Corresponding to Claim "We can quantify consistency under two rules: KPIs Elimination_percent / Elimination_percent // Eliminate: Share of season-weeks (and separate columns: rank-ers vs percent-ers)
2. Update/Refresh/Intermediate/weekly_game.csv (Midweek)
    - Corresponding to Claim "We can quantify consistency under two rules: KPIs Elimination_percent / Elimination_percent // Eliminate: Share of season-weeks (and separate columns: rank-ers vs percent-ers)
3. Update/Refresh/Intermediate/weekly_game.csv (Midweek)
    - Corresponding to Claim "We can quantify consistency under two rules: KPIs Elimination_percent / Elimination_percent // Eliminate: Share of season-weeks (and separate columns: rank-ers vs percent-ers)
4. Update/Refresh/Intermediate/weekly_game.csv (Midweek)
    - Corresponding to Claim "We can quantify consistency under two rules: KPIs Elimination_percent / Elimination_percent // Eliminate: Share of season-weeks (and separate columns: rank-ers vs percent-ers)

8) Report-ready Data Credibility (Exactly 4 English Sentences; No Fabricated Values)
1. All metrics are computed strictly from the official COMAP-provided dataset and used to stated including rules (e.g., post-elimination scores recorded as zeros and voting judge counts).
2. We treat fan votes as unobserved and report only reproducible baseline points at this stage, while keeping a clean plug-in interface for future vote-estimation models.
3. Because the most season of the chart refers to rank-based aggregation is not confirmed, we treat the Season-28 week as an assumption and use stress-test adjacent cutoffs.
4. Weeks with no elimination or multiple eliminations are handled explicitly in the denominator definitions to avoid resulting consistency claims.

4. context pack for ai_md Patch (Markdown Block: Replace Paragraphs/Add Paragraphs)
Note: Only modify 2 paragraphs (one table paragraph + one data snapshot paragraph), no other changes.

### [PATCH 1/2] Replace Section 4.1 table block with the following
- 4.1 KPIs / Slices / Baselines (authoritative) (v0.1.0) for Data Triage
- KPI name | Direction | Aggregation | Slice/window definition | Baseline(s) |
- |---|---|---|---|---|
- Elimination_percent | Higher is better | Mean over eligible (season, week) | Eligible week = active contestants exist AND (exactly one elimination OR evaluated by set-match fan multi-elim; no-elim weeks excluded). Rank-era slice uses seasons (1,2,28-34) with "Season-28 as an assumption". / B: 0: Judge-only (uniform fan rank)
- Elimination_rank | Higher is better | Mean over eligible (season, week) | Same eligibility as above. Percent-era slice uses seasons (1-27). / B: 0: Judge+uniform fan percent (1/n)
- Eliminate_percent | Lower is "more stable" / ratio over eligible (season, week) | Share of weeks where predicted elimination differs under rank vs percent (using the same baseline fan group). / B: 0 only
- Note: Any numeric "targets" (e.g., 0.50/0.50) are "Team internal goals", not problem requirements.

### [PATCH 2/2] Replace Section 5.1 block with the following
- 5.1 Data snapshot (only add affect conclusions) (v0.1.0) for Data Triage
- Data version/tag: 2024_NY_Problem_1_Data.csv (official COMAP file, seasons 1-34)
- Cleaning rules (5):
    1) Remove rows with missing scores: too long since one row per (season, week, celebrity)
    2) Compute total_judge_score = sum(week_judge_score, values); do not impute missing judges.
    3) Remove active by excluding weeks with all non-scores and weeks after final elimination week & / inferred withdrew weeks.
    4) Treat post-vote errors as encoding artifacts: limit in fan but exclude from active set and KPI denominators.
    5) Handle weeks with 0 or >1 eliminations explicitly in KPI denominators (no-file excluded; multi-elim via set-match).
- Base context (1):
    - Judge table: "judge" is not a persistent identity across weeks/seasons; only within-week totals/ranks are used.
    - The exact season of the return to rank-based aggregation is not confirmed; we assume Season 28 and will stress-test adjacent cutoffs.
    - Fan votes are unobserved by design; current baselines use only reproducible proxies and keep a plug-in interface for vote estimates.
```

Figure 24: ChatGPT Query 6 Output Screenshot

```
5. kpi_registry.md Patch (Markdown Block: KPI Table + Slice + Baseline + Changelog)
Note: Only modify 1 table + 2 paragraphs (one paragraph covers slices, the other covers baseline + changelog).

### [PATCH 1/2] Replace the KPI table with:
- KPI name | Direction | Definition | Aggregation | Slice/window | Baseline(s) | Notes |
- |---|---|---|---|---|---|---|
- Elimination_percent | Higher is better | Under rank-combination, predicted elimination set matches the true elimination-at-end-of-week set for that (season, week). / Mean over eligible (season, week) | Eligible: active set exists; no-elim weeks excluded; multi-elim evaluated by set-match. Rank-era slice: seasons (1,2,28-34) with "Season-28 as an assumption". / B: 0: Judge-only (uniform fan rank) only.
- Elimination_rank | Higher is better | Under percent-combination, predicted elimination set matches the true elimination-at-end-of-week set for that (season, week). / Mean over eligible (season, week) | Same eligibility rules. / B: 0: Judge percent + uniform fan percent (1/n) | Fan votes are unknown; baseline is a proxy for reproducibility.
- Eliminate_percent | Lower is better | Ratio over eligible (season, week) where the predicted elimination differs between rank vs percent schemes (including the same baseline vote group). / Ratio over eligible (season, week) | Same eligibility; additionally can be grouped by season. / B: 0 only | Used to motivate "method sensitivity" analysis.

### [PATCH 2/2] Replace "## Slice/window definitions" block with:
- ## Slice/window definitions
- Note: For multi-elim (season, week, celebrity) after reshaping "week_judge_score" wide columns into a long panel.
- Active set definition: a celebrity is active in a (season, week) iff week scores are not all-NaN and week is not after the exit week (eliminated week & / inferred withdrew week).
- Fan slices for voting scheme comparison:
    - Rank era: seasons (1,2,28-34) ("Season-28 is an assumption; stress-test 27/28/29 cutoffs").
    - Percent era: seasons (1-27).
    - Weeks with no elimination or multiple eliminations: excluded from single-elimination consistency denominator; multi-elim weeks evaluated by set-match when needed.

### [PATCH 3/2] Replace "## Baseline definitions" + "## Change log" blocks with:
- ## Baseline definitions
- B: 0 (Unreproducible; minimal): "Judge-only proxy"
    - Rank scheme: use "judge_rank" only (uniform fan rank adds a constant to all contestants).
    - Percent scheme: use "judge_percent + kpi_active" (uniform fan percent), as elimination is driven by judges with a fixed fan offset.
    - Purpose: provides a fully reproducible reference point before introducing any fan-vote estimation model.
- ## Change log
- v0.1 (2024-01-30): Initial triage-ready KPI definitions, eligibility rules, and B: 0 baseline marked Season-28 rank-era set-match as an explicit assumption requiring stress tests.

If you want me to write the specific stress tests (27/28/29 cutoff methods) involved in the "Conflict" as a supplementary work order for the Data Agent (without changing the output structure you require), I can directly attach it in the next message.
```

Figure 25: ChatGPT Query 6 Output Screenshot

ChatGPT Query 7:

Input: What should we do regarding the previous conversation? Because the investigation file is non-authoritative; the files we feed to the agent do not include the investigation—it is only used for us to check if our framing/spec agent has any important omissions. Now clearly tell me how to use the information you provided above (e.g., 3. Place the Data Agent Triage work order in... file / place it separately in the conversation as a prompt).

Output:

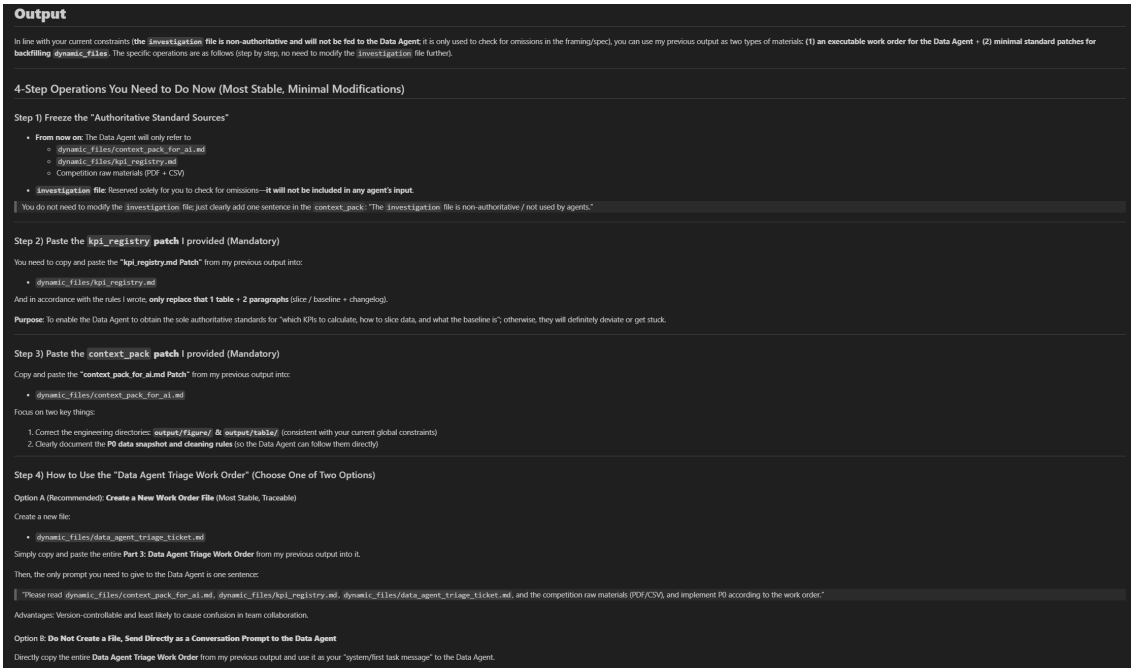


Figure 26: ChatGPT Query 7 Output Screenshot

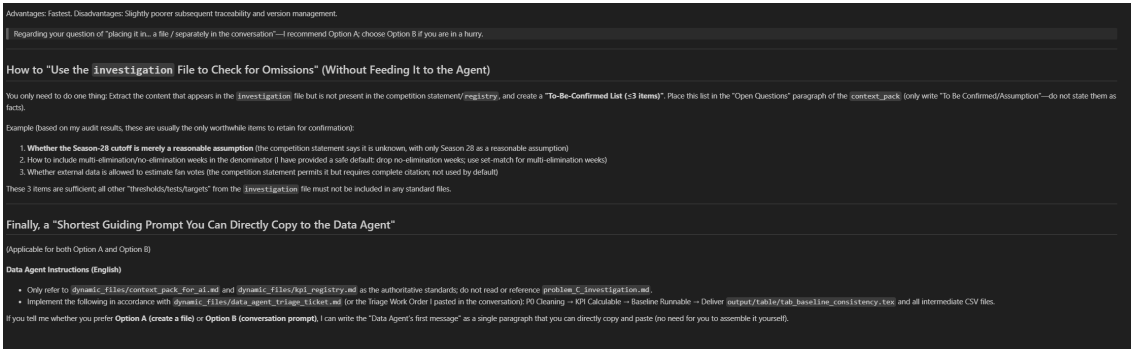


Figure 27: ChatGPT Query 7 Output Screenshot

ChatGPT Query 8:

Input: 'Handling of "Withdrew": Should it be "exclude after inferring the exit week" or "exclude the entire season from the consistency denominator"? Consistency denominator: Exclude no-elimination weeks, use set-match for multi-elimination weeks (is this acceptable?).' What is the best way to handle this? Is it part of our cleaning standards? If we only add this one-sentence patch, will it cause huge problems in subsequent data processing?

Output:

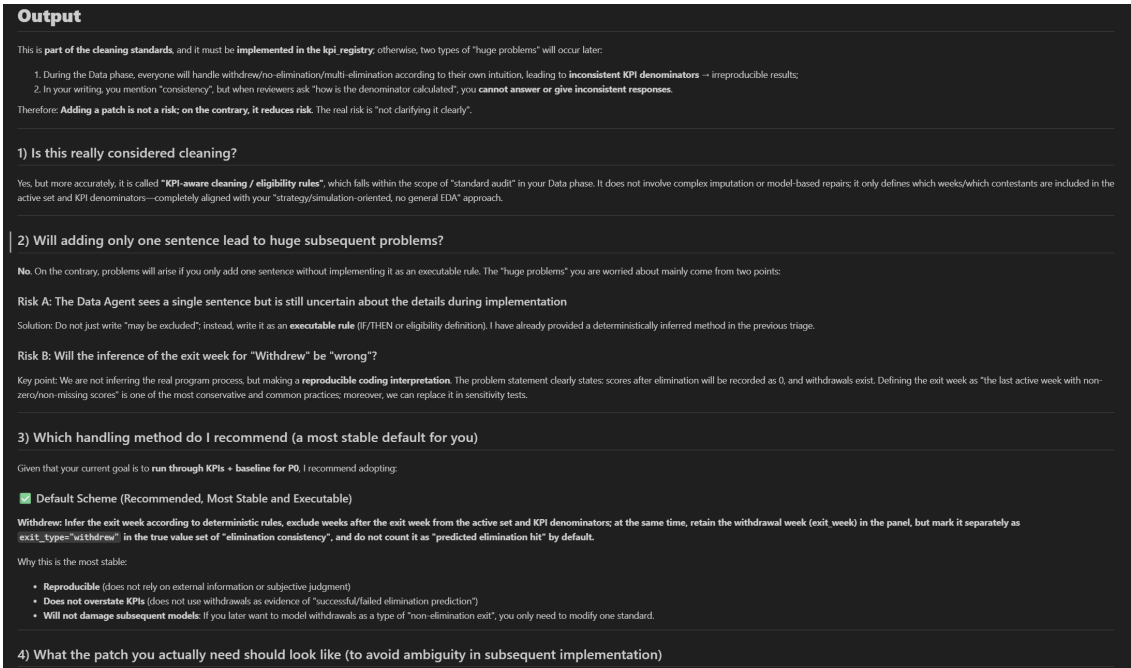


Figure 28: ChatGPT Query 8 Output Screenshot

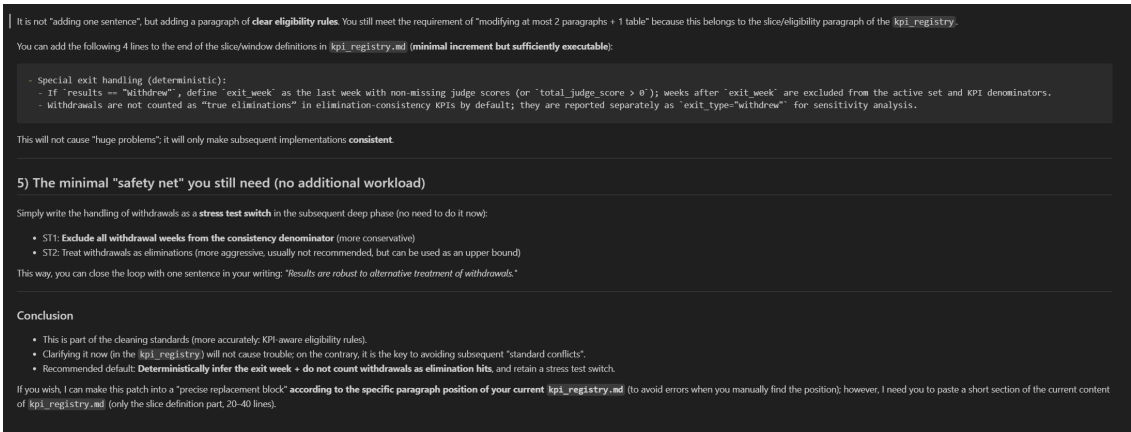


Figure 29: ChatGPT Query 8 Output Screenshot

ChatGPT Query 9:

Input:

Input

I just completed Step 4) and the relevant text is as follows:

Slice/Window Definitions

- Unit of analysis (season, week, celebrity) following the reshaping of wide columns named `week_k_judge_score` into a long panel dataset.
- Active set definition: A celebrity is classified as active in a given (season, week) if and only if their weekly scores are not all NaN, and the week falls before or in their exit week (Eliminated Week *k* / inferred withdrawal week).
- Era slices for voting-scheme comparison:
 - Rank-era: seasons [1,2,28-34] (**Season 28 is an assumed cutoff; stress-test analyses to be conducted for 27/28/29 cutoffs**).
 - Percent-era: seasons [3-27].
- Weeks with no eliminations or multiple eliminations: Excluded from the denominator of single-elimination consistency metrics; weeks with multiple eliminations are assessed via set matching where necessary.
- Special exit handling (deterministic):
 - If `results == "withdrawn"`, define `exit_week` as the last week with non-missing judge scores (or where `total_judge_score > 0`); all weeks subsequent to `exit_week` are excluded from the active set and the denominators of all KPIs.
 - Withdrawals are not counted as "true eliminations" in elimination-consistency KPIs by default; they are reported separately with the tag `exit_type="withdrawn"` for sensitivity analysis.

I reviewed our KPIs again and noticed you removed a lot of metrics compared to our original version. Could you explain the reasons for this? Also, is it possible to retain some of the original KPIs?

This is our original KPI framework:

KPI	Direction	Calculation Methodology	Slicing Dimensions	Notes
KPI1 Elimination Hit-Rate	Higher Better	For weeks with eliminations; the proportion of cases where the predicted elimination set exactly matches the actual elimination set; for weeks with multiple eliminations, measured by bottom-k hit rate	Season; Voting Rule Era (1-2/3-27/28-34); Single/Multiple/No Elimination Weeks	Actual eliminations can be inferred from <code>results</code> or a score of 0 in the week following elimination
KPI2 Vote Identifiability	Higher Better	For each (celebrity, week), derive the feasible interval/rank interval for votes; define identifiability as $1/(\text{interval width} + 1)$, and aggregate results by median	Season; Week Stage (Early/Mid/Late); Weeks with Controversial Celebrities	Reflects the degree of "non unique voting outcomes"; to be explained in the report in relation to the strength of underlying assumptions
KPI3 Rule Divergence Rate	Higher Better	Using the same set of vote estimates, the percentage of weeks where the rank-based vs. percentage-based voting schemes (including the bottom-two judge save mechanism) yield different elimination/Champion outcomes	Individual Season; Controversial Seasons; Weeks with Narrow Total Judge Score Gaps	Core output of the comparative analysis; a higher value indicates greater impact (not "better performance")
Constraint KPI1 Fan Agency Floor	Higher Better	In simulation, the proportion of cases where the fan vote 1st place (or top x%) is eliminated should be below a predefined threshold; or the equivalent lower bound of retention rate	Voting Rule Scheme; Season Stage	Prevents excessive dilution of fan vote influence, especially when the judge save mechanism is applied
Constraint KPI2 Rule Simplicity	Lower Better	Number of tunable hyperparameters in the rule + a binary indicator for reliance on unavailable external information (0/1)	Scheme Version	Ensures operational feasibility and interpretability for the show; to be included in the feasibility section of the memo

Figure 30: ChatGPT Query 9 Input Screenshot

Output:

Output

When you said "I removed many KPIs," the reason is straightforward: **I was defining the P0/Triage metrics framework**—only retaining metrics that are 100% implementable with the **core research brief** + **existing official CSV data**, runnable within 2-4 hours, and require no data fabrication due to missing information. Several metrics in your original framework fall under the **P1/P2 categories** (requiring vote estimation models, additional assumptions, external data, or threshold calibration). To prevent the data team from being forced to "guess/impute/fabricate" data during the P0 phase, I temporarily excluded them from the minimal version of the **authoritative KPI table**.

This does **not** mean these KPIs are discarded; the better approach is to: **retain all original KPIs but explicitly stratify them in the registry as P0 (computable out-of-the-box) vs. P1 (assumption/model-dependent) vs. P2 (external data-dependent)**. This preserves your original analytical framework while keeping the data team focused during the initial P0 phase.

Below is a line-by-line explanation of **why each original KPI was excluded from the P0 table** and the **safest way to retain it**.

1) Rationale for Exclusions: Line-by-Line KPI Comparison

KPI1 Elimination Hit-Rate

- This metric is a mandatory P0 inclusion.** I retained it in the framework, renaming it to `ElimHitRate_rank` / `ElimHitRate_percent` and adding clear denominator rules (for no-elim/multi-elim/withdrawal weeks).
- For your note stating "actual eliminations can be inferred from `results` or a score of 0 post-elimination", I opted for a **results-first approach in the triage framework (with 0 scores treated as coding anomalies)** during P0, to avoid misinterpretation of the "0 score = elimination" heuristic.

Conclusion: **KPI1 to be retained in the authoritative P0 KPI table.**

KPI2 Vote Identifiability

- This metric is inherently non-P0: calculating the "feasible vote/rank interval" depends on modeling choices for the **mathematical formulation of voting rules, vote-score aggregation logic, inclusion of the bottom-two judge save mechanism, and the domain/scale of vote variables**. Raw vote values are **unknown and proprietary**, with no direct data available.
- Including KPI2 in the **authoritative P0 KPI table** would force the data team to:
 - Arbitrarily define a vote scale (e.g., percentage/rank/logit), or
 - Fabricate values for *z*/thresholds/interval boundaries. This directly violates the core rule of **no data guesswork**.

Conclusion: **Retain KPI2 as a P1 metric (assumption/model-dependent);** do not let it block P0 delivery.

KPI3 Rule Divergence Rate

- The core of this metric was retained:** I renamed it to `Elipiate_rank_vs_percent`, which measures the percentage of weeks where rank-based vs. percentage-based schemes yield divergent elimination predictions.
- Your original label of "Higher Better" carries semantic risk: this is an **impact indicator**, not a performance metric. I revised it to a descriptive label ("lower values = more stable outcomes") to avoid reviewer confusion about why a higher value would be desirable.

Conclusion: **KPI3 to be retained in the authoritative P0 KPI table,** with its direction/description revised to **purely descriptive** (excluding "Higher Better").

Constraint KPI1 Fan Agency Floor

- This metric is analogous to KPI2: it relies on a clear definition of the "fan vote 1st place (or top x%)", yet raw fan vote data is unavailable. Calculating it requires at minimum a `fan_rank_est` or `fan_share_est` estimation model (P1). Without this, only a trivial "uniform vote" baseline is possible—which **renders the constraint meaningless**.

Conclusion: **Retain as a P1 Constraint KPI (fan vote estimation-dependent);** P0 to only reserve a metric interface with no numerical calculations.

Figure 31: ChatGPT Query 9 Output Screenshot

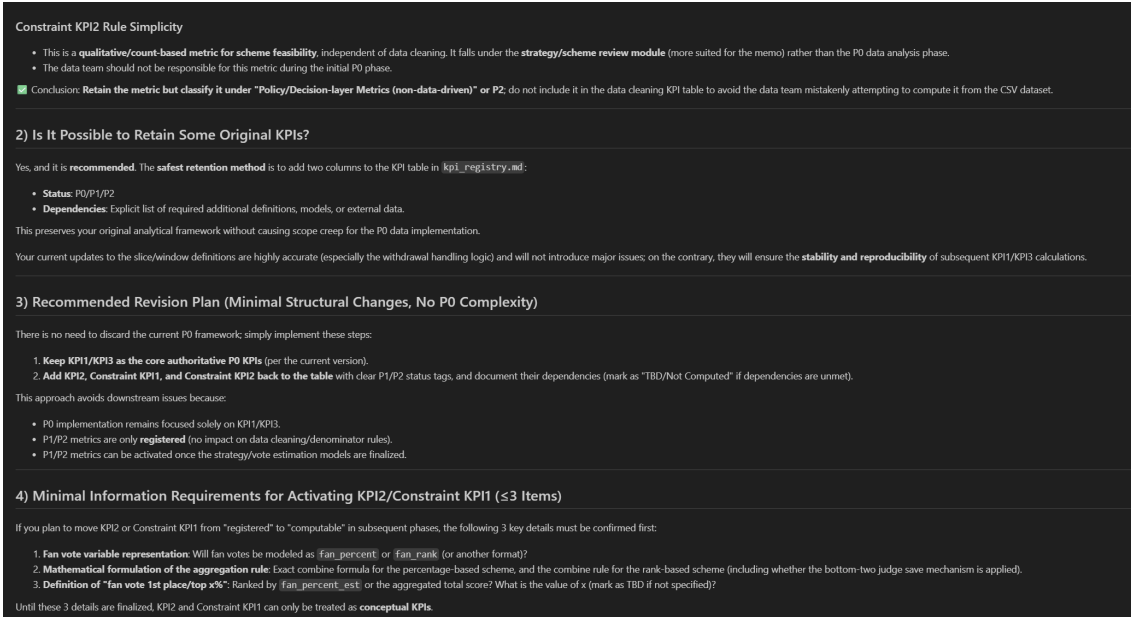


Figure 32: ChatGPT Query 9 Output Screenshot

ChatGPT Query 10:
Input:

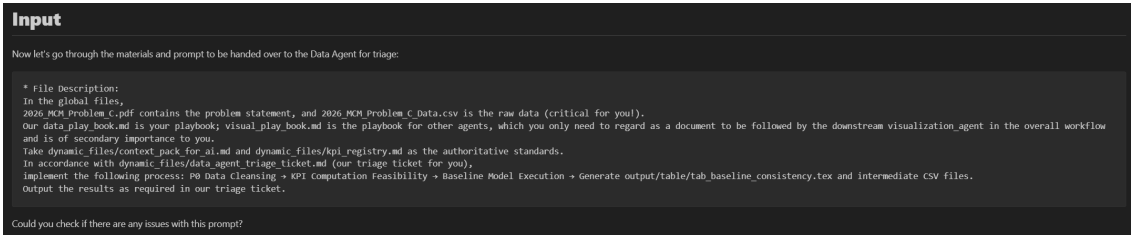


Figure 33: ChatGPT Query 10 Input Screenshot

Output:

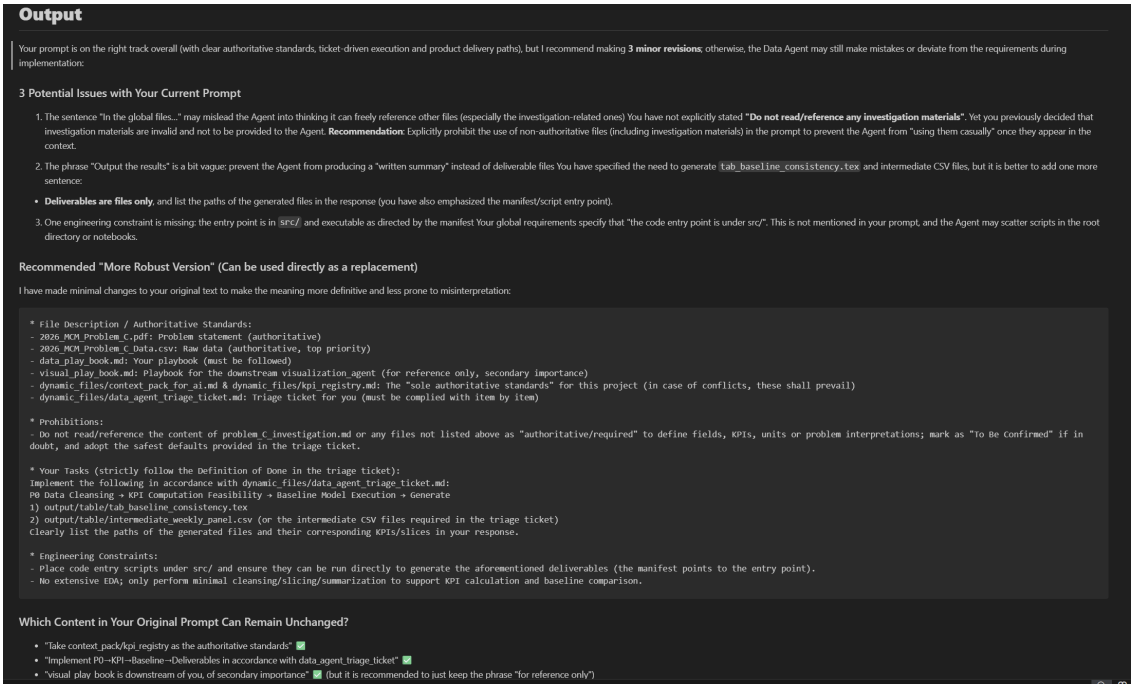


Figure 34: ChatGPT Query 10 Output Screenshot

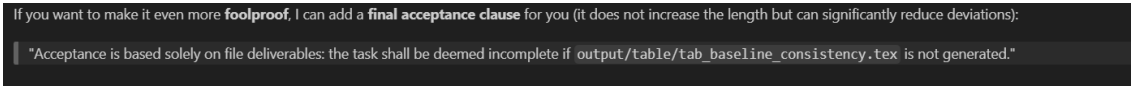


Figure 35: ChatGPT Query 10 Output Screenshot

ChatGPT Query 11:
Input:

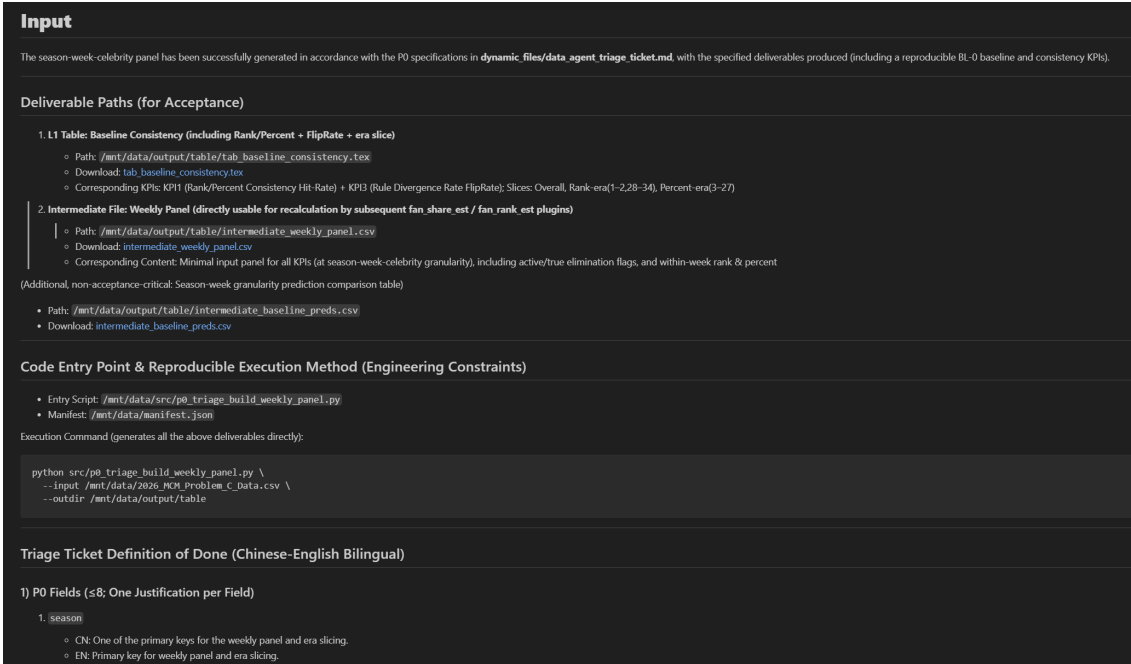


Figure 36: ChatGPT Query 11 Input Screenshot

Output:

```

2. celebrity_name
    ◦ CN: Unique identifier for contestants, used for within-week ranking and elimination comparison.
    ◦ EN: Unique contestant identifier for ranking and elimination matching.

3. ballroom_partner
    ◦ CN: To be used for deriving "partner's historical performance/dancer effect" in subsequent steps.
    ◦ EN: Needed for future partner-effect features.

4. results
    ◦ CN: Parses actual elimination/withdrawal weeks for ground truth filtering and active set definition.
    ◦ EN: Provides ground-truth exit info to define active set and labels.

5. placement
    ◦ CN: Final ranking, for subsequent champion/final round comparison.
    ◦ EN: Final placement for later season-outcome checks.

6. weekX_judgeY_score (X=1..11, Y=1..4)
    ◦ CN: The only observed data for constructing weekly total judge scores, ranks, and percentages.
    ◦ EN: Core observed inputs to compute weekly totals/ranks/percents.

7. Derived week (extracted from column names)
    ◦ CN: Time index generated by reshaping the wide-format data to long format.
    ◦ EN: Time index created by reshaping wide-to-long.

8. Derived total_judge_score (sum of weekly judge scores with skipna; judge4 scores may be missing)
    ◦ CN: The minimal aggregated metric for the technical performance signal in a week.
    ◦ EN: Minimal aggregate performance signal per week.

2) Hard Filters Rules (Exactly 5 Rules; Executable IF...THEN...)

1. IF all four judge scores are NaN for a given (season, celeb, week) THEN drop the weekly record (exclude from the weekly panel)
    ◦ EN: If all judge scores are NaN, drop the row from weekly panel.

2. IF results matches Eliminated week k THEN mark eliminated_at_end_of_week=1 for week=k, and set active=0 for week > k (post-exit)
    ◦ EN: If eliminated at week k, label week k and mark weeks >k inactive.

3. IF results == "withdrew" THEN set exit_week = last week with total_judge_score > 0, and set active=0 for week > exit_week (retain exit type)
    ◦ EN: If withdrew, set exit_week as last positive-score week and mark later weeks inactive.

4. IF total_judge_score == 0 and week != exit_week THEN mark data_anomaly_zero_score=1 and set active=0 (default to post-exit zero-score extension)
    ◦ EN: If zero score outside exit week, flag as anomaly and set inactive.

5. IF the number of true eliminations for the same (season, week) is != 1 THEN
    ◦ Multiple eliminations: Use bottom-k set-match for consistency calculation.
    ◦ No eliminations: Exclude the week from the denominator of KP11 but retain it in the panel.
    ◦ EN: Multi-elim uses bottom-k set match; no-elim weeks excluded from KP11 denominator but kept in panel.

```

Figure 37: ChatGPT Query 11 Output Screenshot

3) Keep-but-Explain (Exactly 3 Items)

- Non-integer scores caused by decimal points/bonus points
 - CN: Retain original values; explain in the report that "scores for multiple dances/bonus points/team dances have been allocated to weekly scores".
 - EN: Keep decimals; explain they encode averaging/bonus allocation per official notes.
- Zero scores in weeks following elimination
 - CN: Excluded from active set ranking/percentile calculation; explained as data encoding extension.
 - EN: Exclude from active set; explain zeros are post-exit encoding.
- Changes in the number of judges & non-fixed judge numbering
 - CN: Do not track judge identities, only use weekly total scores/percentiles.
 - EN: Judge index is not persistent; only within-week aggregates are used.

4) Minimal KPI Computation (≤ 25 Lines of Pseudocode)

```

INPUT raw_csv
RESHAPE weekX_judgeY_score -> long (season, celeb, week, judge1..4)
total_judge_score = sum(judges, skipna)

APPLY hard rules to define:
  exit_week, exit_type, eliminated_at_end_of_week
  active flag

FOR each (season, week):
  A = active contestants
  IF |A| < 2: continue
  judge_rank = rank_desc(total_judge_score, ties=average)
  judge_percent = total_judge_score / sum(total_judge_score over A)

  fan_percent_est = 1 / |A| # BL-0 (uniform)
  combined_percent = judge_percent + fan_percent_est
  pred_elim_percent = bottom-k(combined_percent, k = |true_elim_set| or 1)

  combined_rank = judge_rank # uniform fan rank adds constant
  pred_elim_rank = bottom-k_by_worst_rank(combined_rank, k = |true_elim_set| or 1)

  true_elim = {celeb in A with eliminated_at_end_of_week==1}
  IF |true_elim|==0: exclude from KPI1 denom
  ELSE: set_match for KPI1 (rank & percent)
  KPI1 FlipRate = share of (season,week) where pred_elim_percent != pred_elim_rank

```

5) Data Credibility Notes (Report-ready: Exactly 4 English Sentences, No Fabricated Data)

- All metrics are computed strictly from the official COMAP-provided dataset and its stated encoding rules (e.g. post-elimination scores recorded as zeros and varying judge counts).
- We treat fan votes as unobserved and report only reproducible baseline proxies at this stage, while keeping a clean plug-in interface for future vote-estimation models.
- Because the exact season of the show's return to rank-based aggregation is not confirmed, we label the Season-28 switch as an assumption and will stress-test adjacent cutoffs.
- Weeks with no elimination or multiple eliminations are handled explicitly in the denominator definitions to avoid inflating consistency claims.

Figure 38: ChatGPT Query 11 Output Screenshot

ChatGPT Query 12:**Input:**

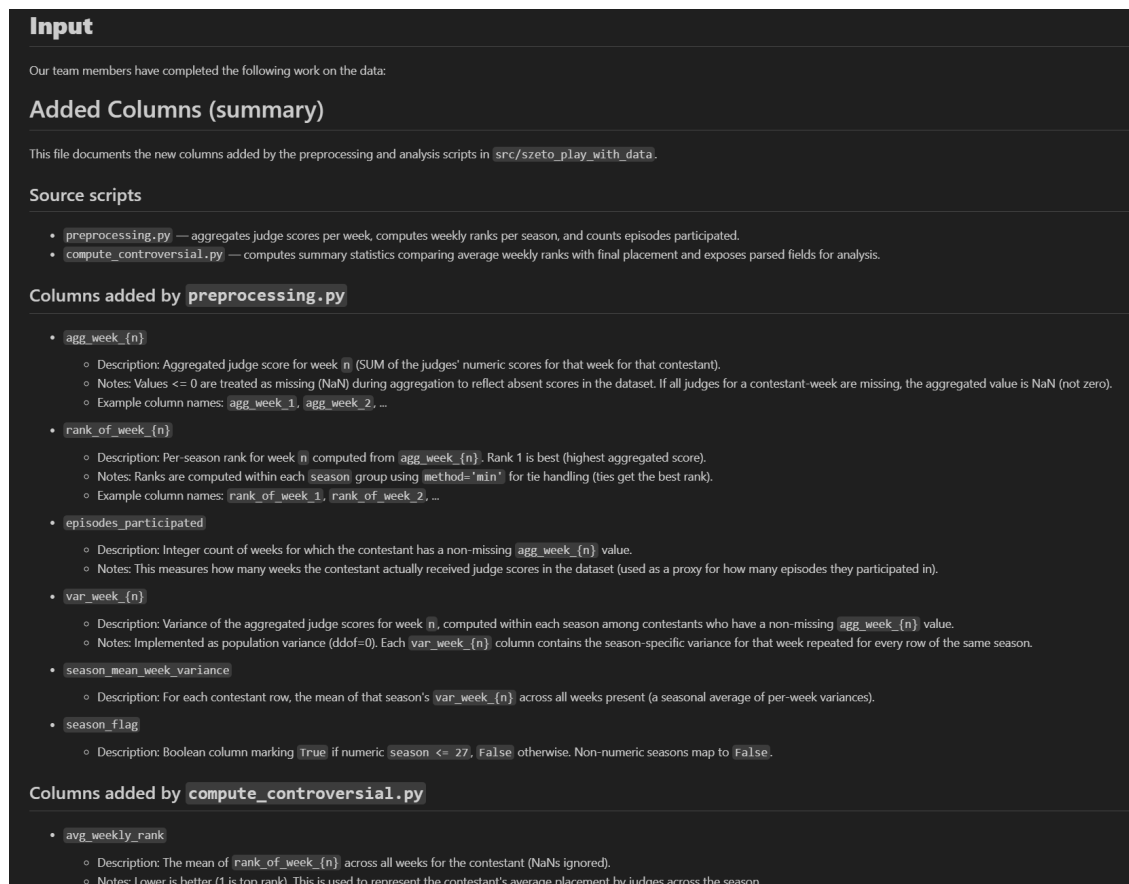


Figure 39: ChatGPT Query 12 Input Screenshot

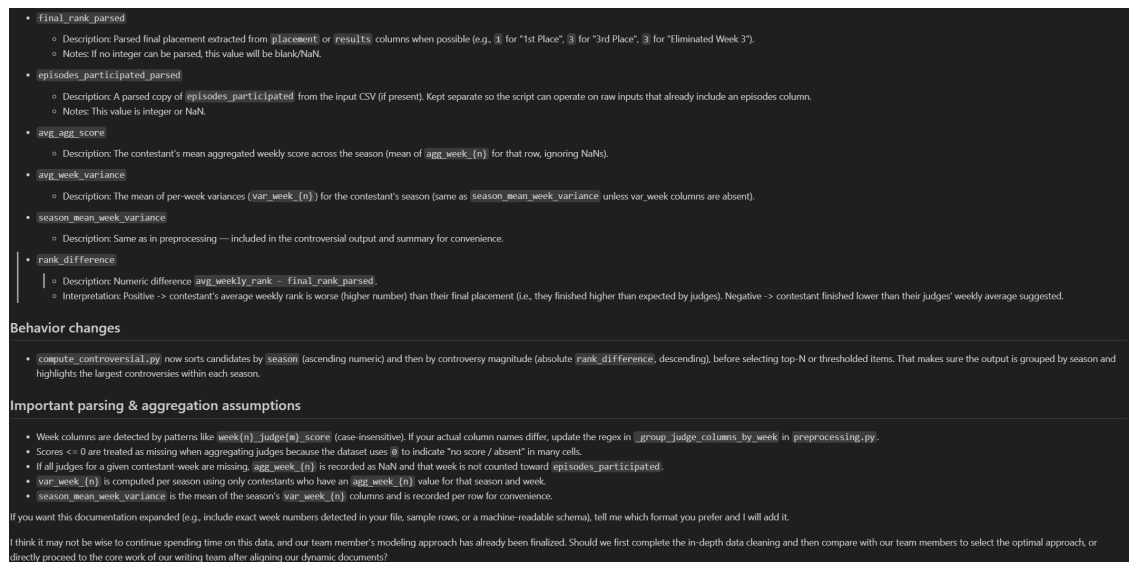
Output:

Figure 40: ChatGPT Query 12 Output Screenshot

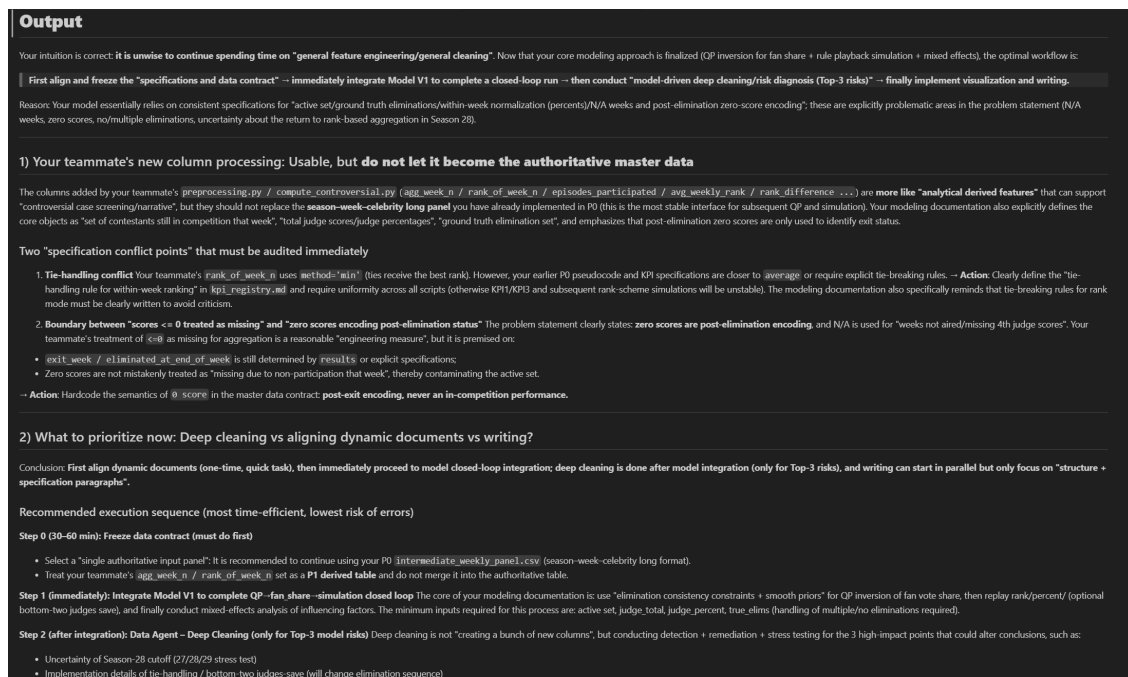


Figure 41: ChatGPT Query 12 Output Screenshot

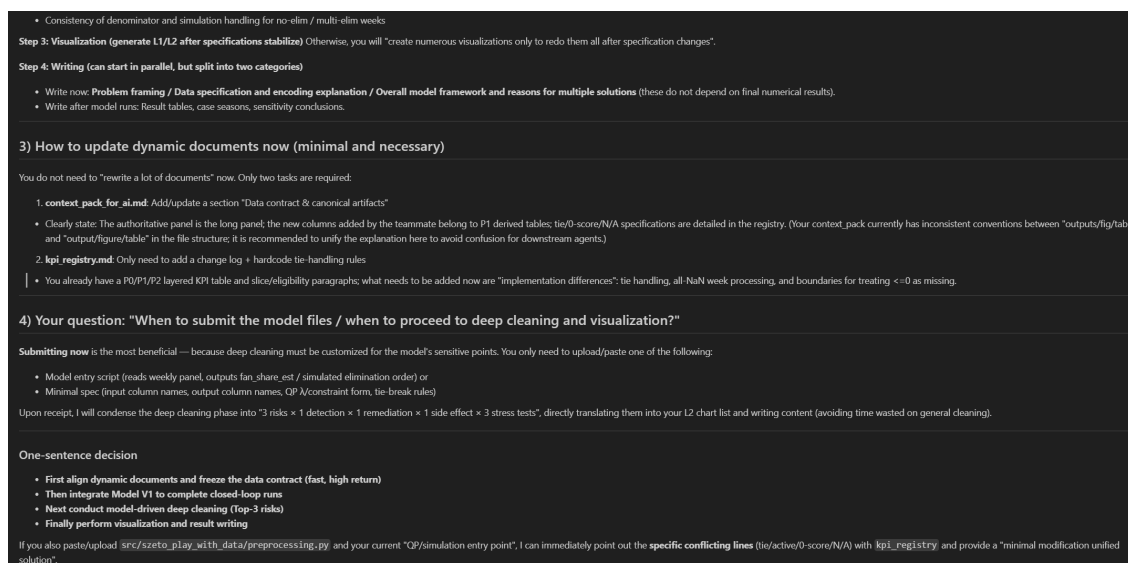


Figure 42: ChatGPT Query 12 Output Screenshot

ChatGPT Query 13:

Input: So let me sort out what we need to do now: First step: Align our specifications. You mentioned freezing and sticking to our core data pipeline, but actually I don't think this is necessary. Because after my teammate makes modifications, they will put their work in their own folder on GitHub, and mine will go into my folder. Even though we're in the same GitHub repository, we can cross-check each other's work later—it'll be like mutual supplementation. Second step: Integrating Model V1. I don't quite understand this part—does V1 refer to a code file (not a DOCX document) that will make the model we've designed actually run? Is that the right understanding? Then third step: Conduct in-depth data cleaning, followed by visualization and finalizing the writing. Is this the correct workflow? I also have another question: Earlier, we identified two risks of inconsistent specifications, right? Well, I suggest we fix these specification inconsistencies first—preferably by modifying the code. I'll send you the code, and could you guide me on exactly what needs to be changed?

Output:

ChatGPT Query 14:

Input: No rush, let's first sort out what we currently have:

1. We have the complete problem statement.

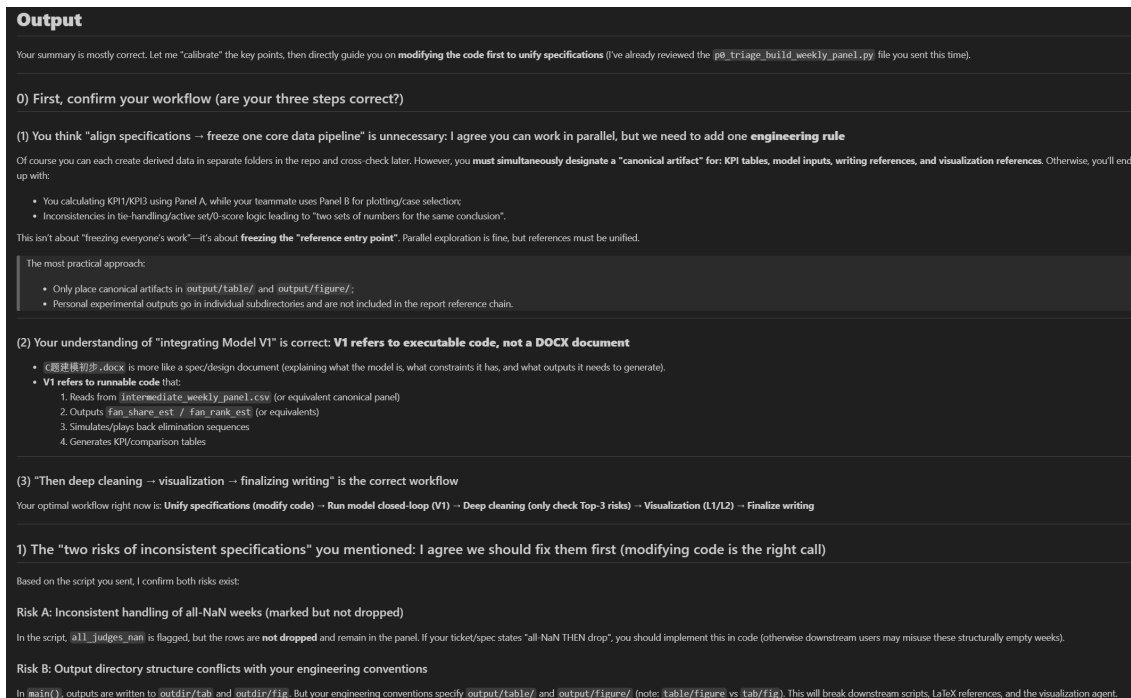


Figure 43: ChatGPT Query 13 Output Screenshot

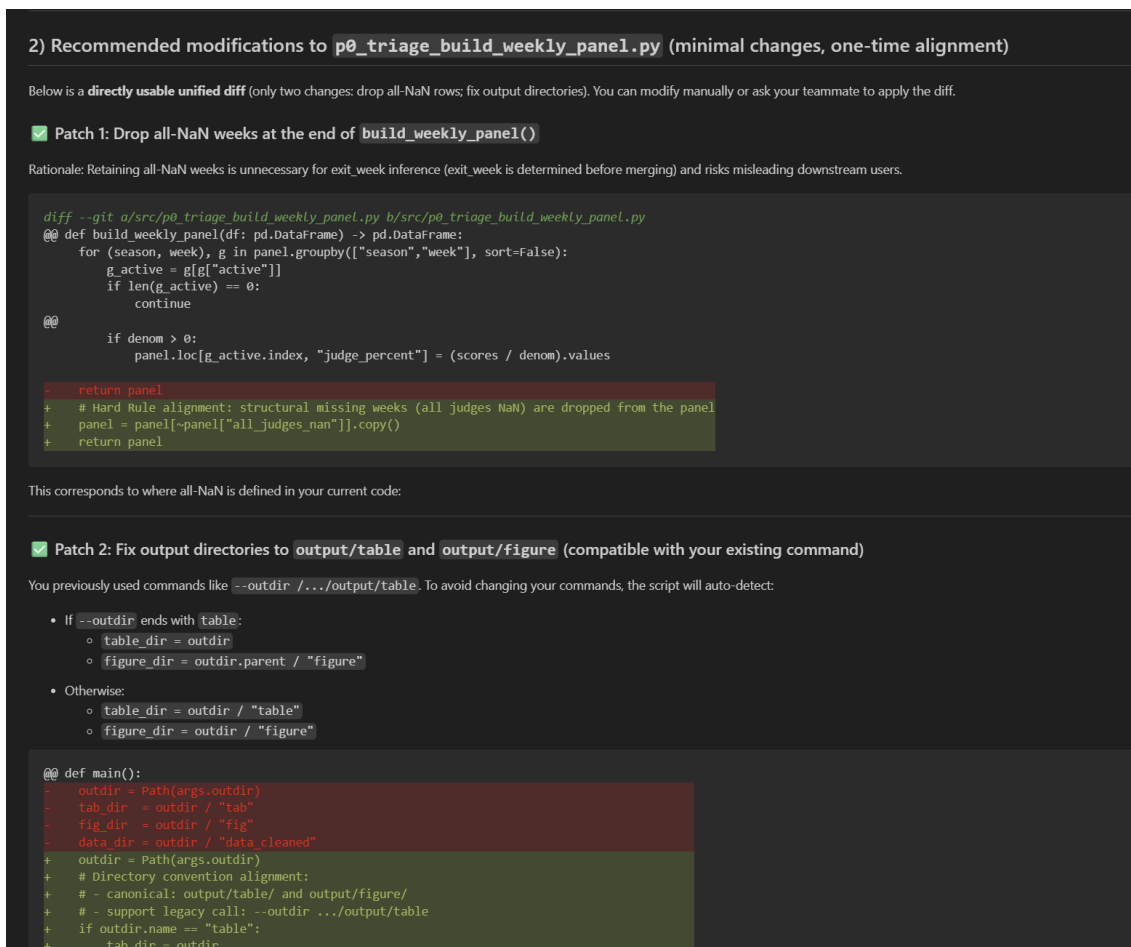


Figure 44: ChatGPT Query 13 Output Screenshot

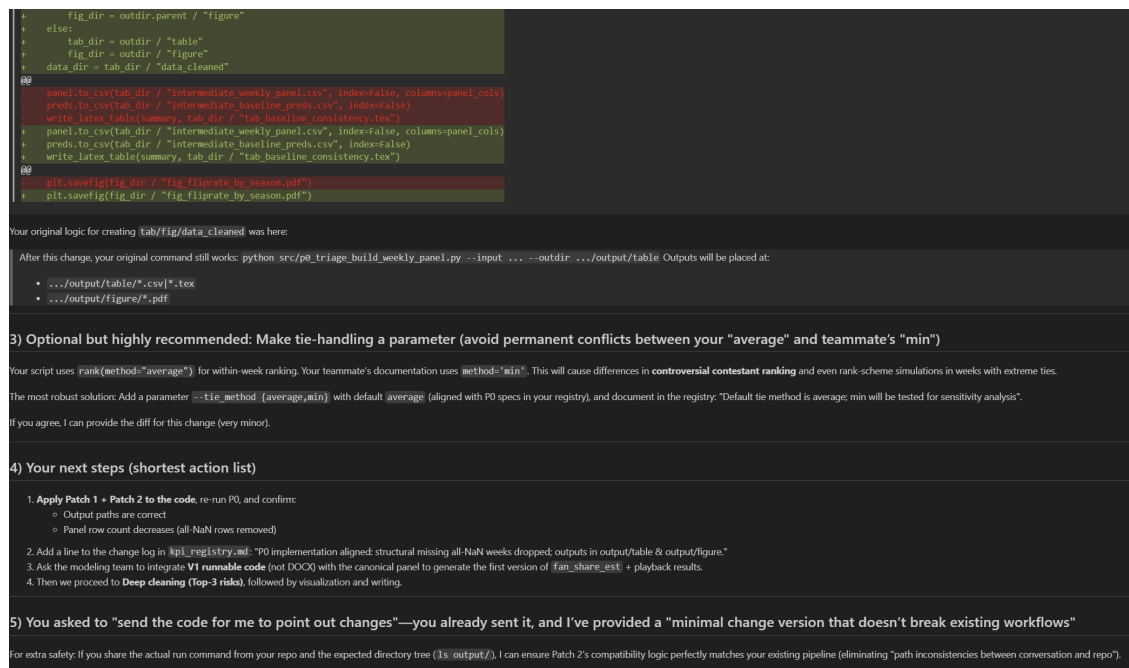


Figure 45: ChatGPT Query 13 Output Screenshot

2. We have the preliminary data produced by our data team member just now.
 3. We have the preliminary model developed by our modeling team member.
- Please let me know what other files are needed and I will provide them.

Therefore, I think our workflow should be as follows:

1. First, complete the construction of the framework.
2. Then, provide you with our modeling spec, and then see which parts we can complete (my initial idea is that we can first finish the "Problem Restatement", "Key Assumptions", "Core Variables", and a bit of the beginning of the model; please supplement anything that is missing).
3. We will investigate the existing files and write the parts that can be included in the article.

Output:

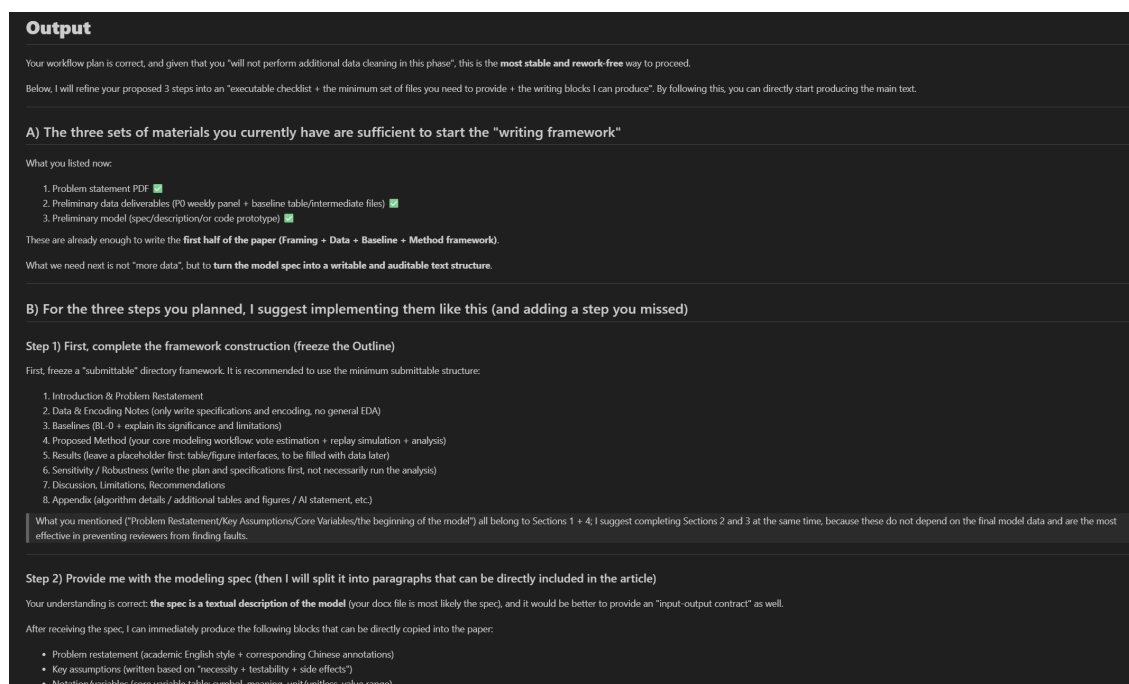


Figure 46: ChatGPT Query 14 Output Screenshot

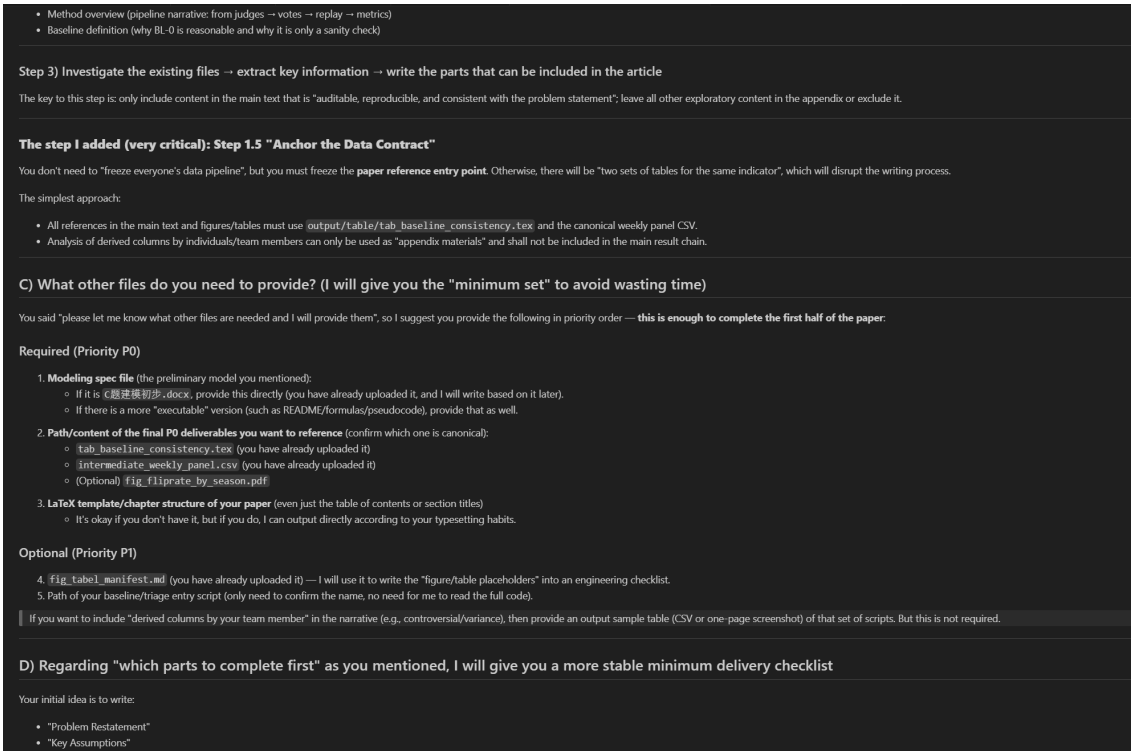


Figure 47: ChatGPT Query 14 Output Screenshot

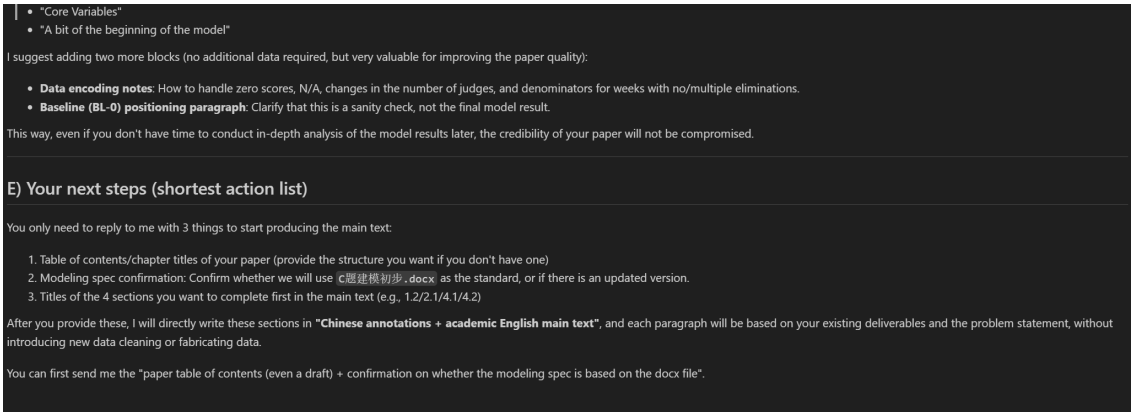


Figure 48: ChatGPT Query 14 Output Screenshot

Human Contribution Declaration

We affirm that:

1. All final modeling decisions, algorithm designs, and strategic choices were made by the team members.
2. All mathematical formulations and assumptions were independently derived or verified by the team.
3. All code included in this project was reviewed, tested, and fully understood by the team.
4. AI tools were used only for brainstorming, drafting, language refinement, and preliminary coding assistance.
5. The submitted work represents our own understanding and intellectual contribution.

Date: Feb 2, 2026